

Laporan Tugas Besar Milestone 3

IF2224 Teori Bahasa Formal dan Otomata

Semantic Analysis

Semester II Tahun 2025/2026



Disusun oleh:

13524041 - Nathan Adhika Santosa

13524053 - Muhammad Haris Putra Sulastianto

13524085 - Ariel Cornelius Sitorus

13524091 - Vara Azzara Ramli Pulukadang

Laboratorium Ilmu Rekayasa dan Komputasi

Sekolah Teknik Elektro dan Informatika

Program Studi Teknik Informatika

Institut Teknologi Bandung

Daftar Isi

1	Teori Singkat	5
1.1	Semantic Analysis	5
1.2	Abstract Syntax Tree (AST)	5
1.3	Decorated Abstract Syntax Tree	6
1.4	Symbol Table	6
1.5	Scope dan Lexical Level	7
1.6	Type Checking	8
1.7	Type Compatibility	8
1.8	Syntax-Directed Translation (SDT)	9
1.9	Attributed Grammar	9
1.10	Semantic Visitor / Visitor Pattern	10
2	Perancangan dan Implementasi	11
2.1	Bahasa dan Library yang Digunakan	11
2.2	Struktur Program	11
2.3	Alur Kerja Semantic Analysis	13
2.4	Desain Abstract Syntax Tree	15
2.5	Representasi Decorated AST	16
2.6	Implementasi Syntax-Directed Translation	17
2.7	Semantic Action yang Digunakan	18
2.8	Implementasi Symbol Table	20
2.9	Implementasi Scope Management	22
2.10	Implementasi Type Compatibility Rule	23
2.11	Implementasi Type Checking	25
2.12	Implementasi Semantic Visitor	26
2.13	Implementasi Error Handling Semantic	27
2.14	Implementasi Output Semantic Analysis	29
2.15	Fungsi dan Kelas Utama	29
2.16	Justifikasi Implementasi	30
3	Type Compatibility Rule	31
3.1	Primitive Type Rule	31
3.1.1	Integer	31

3.1.2	Real	32
3.1.3	Boolean	32
3.1.4	Char	33
3.1.5	String	33
3.2	Structured Type Rule	34
3.2.1	Array	34
3.2.2	Record	34
3.2.3	Enumerated	35
3.2.4	Subrange	36
3.3	Assignment Compatibility	36
3.4	Operator Compatibility	37
3.4.1	Arithmetic Operator	37
3.4.2	Relational Operator	38
3.4.3	Logical Operator	39
3.5	Scope Rule	40
3.6	Function dan Procedure Rule	41
3.7	Ringkasan Type Compatibility Rule	42
4	Hasil Implementasi	42
4.1	Hasil Konversi Parse Tree ke AST	43
4.2	Hasil Decorated AST	44
4.3	Hasil Symbol Table	44
4.3.1	Contoh Tab	44
4.3.2	Contoh Btab	45
4.3.3	Contoh Atab	45
4.4	Hasil Type Checking	46
4.5	Hasil Scope Checking	46
4.6	Hasil Semantic Error Detection	47
4.6.1	Undeclared Identifier	47
4.6.2	Redeclaration	48
4.6.3	Type Mismatch	48
4.6.4	Invalid If Condition	48
4.6.5	Invalid Range	49
4.7	Ringkasan Hasil Implementasi	49

5	Pengujian	49
5.1	Ringkasan Test Case	50
5.2	Test Case 1: Program Valid Sederhana	50
5.2.1	Tujuan Pengujian	50
5.2.2	Input Program	50
5.2.3	Output Program	51
5.2.4	Hasil Pengujian	52
5.3	Test Case 4: Redeclaration pada Scope yang Sama	52
5.3.1	Tujuan Pengujian	52
5.3.2	Input Program	52
5.3.3	Output Program	53
5.3.4	Hasil Pengujian	53
5.4	Test Case 5: Array dan Indexing	54
5.4.1	Tujuan Pengujian	54
5.4.2	Input Program	54
5.4.3	Output Program	54
5.4.4	Hasil Pengujian	55
5.5	Test Case 7: Record dan Akses Field	55
5.5.1	Tujuan Pengujian	55
5.5.2	Input Program	55
5.5.3	Output Program	56
5.5.4	Hasil Pengujian	57
5.6	Test Case 9: Procedure dengan Parameter	57
5.6.1	Tujuan Pengujian	57
5.6.2	Input Program	57
5.6.3	Output Program	58
5.6.4	Hasil Pengujian	58
5.7	Analisis Hasil Pengujian	59
6	Kesimpulan	60
A	Lampiran	61
A.1	Grammar dan Semantic Action	61
A.2	Struktur AST	61
A.3	Contoh Decorated AST	62

A.4 Contoh Symbol Table Lengkap	62
A.5 Pranala Ke Repositori	63
A.6 Pembagian Tugas	63
Referensi	64

1 Teori Singkat

1.1 Semantic Analysis

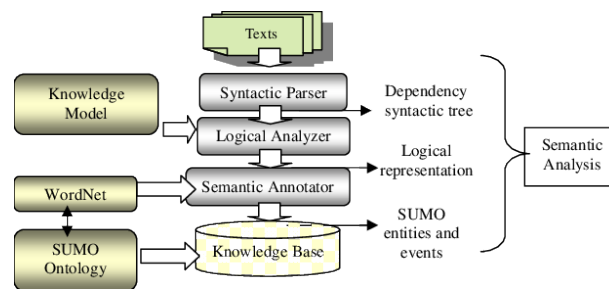


Figure 1: Semantic Analysis Pipeline [4]

Semantic Analysis adalah tahap pada compiler yang memeriksa apakah program sudah benar secara makna. Pada tahap sebelumnya, parser hanya memastikan bahwa susunan token sudah sesuai grammar. Namun, program yang benar secara syntax belum tentu benar secara semantic.

Contoh:

```

1 program Contoh;
2 var
3   x : integer;
4 begin
5   x := 'halo';
6 end.
  
```

Program tersebut memiliki bentuk assignment yang benar secara syntax. Namun secara semantic program tersebut salah, karena variabel `x` bertipe `Integer`, sedangkan nilai yang diberikan bertipe `String`.

Pada semantic analysis, beberapa hal yang dicek antara lain:

1. apakah identifier sudah dideklarasikan sebelum digunakan,
2. apakah identifier dideklarasikan ulang pada scope yang sama,
3. apakah tipe data pada assignment dan expression compatible,
4. apakah penggunaan scope, procedure, function, array, dan record valid.

Pada milestone ini, semantic analysis menerima masukan berupa parse tree dari parser. Parse tree tersebut kemudian dikonversi menjadi AST dan dianotasi menjadi Decorated AST dengan bantuan symbol table.

1.2 Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) adalah bentuk tree yang lebih ringkas dari parse tree. Jika parse tree masih menyimpan struktur grammar secara lengkap, AST hanya menyimpan bagian yang penting terhadap makna program.

Sebagai contoh, statement berikut:

```
1 x := a + b * 5;
```

Pada parse tree, statement tersebut akan memiliki banyak node grammar seperti `<expression>`, `<simple-expression>`, `<term>`, dan `<factor>`. Pada AST, struktur tersebut cukup direpresentasikan berdasarkan hubungan operator dan operand:

```
1 AssignNode
2 |-- VarNode(x)
3 '-- BinOpNode(+)
4     |-- VarNode(a)
5     '-- BinOpNode(*)
6         |-- VarNode(b)
7         '-- NumberNode(5)
```

Dengan bentuk seperti ini, semantic analyzer dapat langsung membaca makna program: variabel `x` diisi dengan hasil expression `a + (b * 5)`. Token syntax seperti `:=`, semicolon, dan node grammar perantara tidak perlu lagi disimpan sebagai node utama.

1.3 Decorated Abstract Syntax Tree

Decorated AST adalah AST yang sudah diberi informasi tambahan dari hasil semantic analysis. Informasi tambahan tersebut dapat berupa tipe data, lexical level, status valid/error, dan referensi ke symbol table.

Contoh AST biasa:

```
1 AssignNode
2 |-- VarNode(b)
3 '-- BinOpNode(+)
4     |-- VarNode(a)
5     '-- NumberNode(10)
```

Setelah proses semantic analysis, node-node tersebut dapat diberi anotasi:

```
1 AssignNode [valid: true]
2 |-- VarNode(b) [type: integer, lev: 0, tab_ref: 39]
3 '-- BinOpNode(+) [type: integer]
4     |-- VarNode(a) [type: integer, lev: 0, tab_ref: 38]
5     '-- NumberNode(10) [type: integer]
```

Dari anotasi tersebut, terlihat bahwa `a` dan `b` sudah ditemukan pada symbol table, keduanya bertipe `Integer`, dan hasil operasi `a + 10` juga bertipe `Integer`. Karena itu, assignment dinyatakan valid.

Jika terdapat error semantic, node dapat diberi anotasi error. Misalnya pada operasi `s + 10` ketika `s` bertipe `String`, node `BinOpNode(+)` dapat diberi tipe `error` karena operator `+` tidak valid untuk kombinasi `String` dan `Integer`.

1.4 Symbol Table

Symbol table adalah struktur data yang menyimpan informasi identifier pada program. Identifier dapat berupa variabel, konstanta, tipe data, procedure, function, parameter, maupun field record.

Contoh deklarasi:

```
1 var
2   a : integer;
3   b : real;
```

Dari deklarasi tersebut, semantic analyzer dapat menyimpan informasi seperti:

```
1 tab
2 idx  identifier  obj      type      lev
3 -----
4 38   a           variable integer    0
5 39   b           variable integer    0
```

Symbol table dipakai ketika identifier digunakan. Misalnya pada statement `a := 10`, semantic analyzer akan mencari `a` pada symbol table untuk memastikan bahwa identifier tersebut sudah dideklarasikan dan mengetahui tipe datanya.

Pada milestone ini, symbol table dibagi menjadi tiga tabel:

1. `tab`, untuk menyimpan informasi identifier;
2. `atab`, untuk menyimpan informasi array;
3. `btabs`, untuk menyimpan informasi block, procedure, function, dan record.

Detail atribut masing-masing tabel dijelaskan lebih lanjut pada bagian perancangan dan implementasi.

1.5 Scope dan Lexical Level

Scope adalah batas area tempat suatu identifier dapat digunakan. Identifier yang dideklarasikan di dalam suatu block hanya berlaku pada block tersebut dan block turunannya.

Contoh:

```
1 program ContohScope;
2 var
3   x : integer;
4
5 procedure Test;
6 var
7   y : integer;
8 begin
9   x := 10;
10  y := 20;
11 end;
12
13 begin
14   x := 5;
15 end.
```

Pada contoh tersebut, `x` berada pada scope global sehingga dapat digunakan di dalam procedure `Test`. Sebaliknya, `y` hanya berada pada scope procedure `Test`, sehingga tidak dapat digunakan langsung pada program utama.

Lexical level menunjukkan kedalaman scope. Scope global berada pada level 0, procedure atau function di dalamnya berada pada level 1, dan block yang lebih dalam memiliki level berikutnya. Informasi ini digunakan saat lookup identifier dari scope terdalam ke scope terluar.

1.6 Type Checking

Type checking adalah proses memeriksa kesesuaian tipe data pada expression, assignment, dan statement. Proses ini diperlukan karena kesalahan tipe biasanya tidak dapat dideteksi hanya dari grammar.

Contoh valid:

```
1 var
2   x : integer;
3 begin
4   x := 10;
5 end.
```

Contoh tidak valid:

```
1 var
2   x : integer;
3 begin
4   x := 'halo';
5 end.
```

Pada contoh kedua, assignment salah karena `x` bertipe `Integer`, sedangkan nilai yang diberikan bertipe `String`.

Type checking juga digunakan pada operator. Misalnya operator aritmatika seperti `+` dan `*` membutuhkan operand numerik, operator logika seperti `and` dan `or` membutuhkan operand `Boolean`, dan kondisi pada `if` atau `while` harus menghasilkan tipe `Boolean`.

1.7 Type Compatibility

Type compatibility adalah aturan yang menentukan apakah dua tipe data boleh digunakan bersama. Aturan ini dipakai oleh proses type checking.

Contoh:

```
1 var
2   a : integer;
3   b : real;
4 begin
5   b := a;
6 end.
```

Assignment tersebut dapat dianggap valid karena nilai `Integer` dapat dimasukkan ke variabel bertipe `Real`. Namun assignment sebaliknya, yaitu memasukkan `Real` ke `Integer`, tidak selalu valid tanpa konversi eksplisit.

Secara umum, aturan compatibility yang digunakan adalah:

1. tipe yang sama dianggap compatible;

2. **Integer** dapat digunakan pada **Real**;
3. **Boolean** hanya compatible dengan **Boolean**;
4. operator relasional menghasilkan **Boolean**;
5. operator logika membutuhkan operand **Boolean**;
6. array dicek berdasarkan tipe index dan tipe elemennya;
7. record dicek berdasarkan aturan named record dan anonymous record.

Aturan lengkap type compatibility yang digunakan pada program dijelaskan pada bagian tersendiri agar tidak membuat teori singkat terlalu panjang.

1.8 Syntax-Directed Translation (SDT)

Syntax-Directed Translation (SDT) adalah teknik untuk menerjemahkan struktur syntax menjadi struktur lain dengan menambahkan semantic action pada aturan grammar. Pada milestone ini, SDT digunakan untuk mengubah parse tree menjadi AST.

Contoh aturan grammar:

```
1 <assignment-statement> -> <variable> becomes <expression>
```

Semantic action-nya dapat ditulis sebagai:

```
1 assignment.node = new AssignNode(variable.node, expression.node)
```

Dengan cara ini, token **becomes** tidak perlu menjadi node utama pada AST. Informasi assignment sudah terwakili oleh **AssignNode**.

Contoh hasil translation:

```
1 <assignment-statement>
2 |-- <variable>
3 |   '-- ident(x)
4 |-- becomes
5 '-- <expression>
6   '-- intcon(10)
```

menjadi:

```
1 AssignNode
2 |-- VarNode(x)
3 '-- NumberNode(10)
```

1.9 Attributed Grammar

Attributed Grammar adalah grammar yang dilengkapi dengan atribut dan aturan semantic. Atribut digunakan untuk membawa informasi tambahan seperti tipe data, nilai constant, scope, atau referensi symbol table.

Contoh:

```
1 <factor> -> intcon
```

Aturan semantic-nya dapat berupa:

```
1 factor.type = Integer
2 factor.value = intcon.value
```

Artinya, ketika parser menemukan `intcon`, node tersebut tidak hanya valid secara syntax, tetapi juga memiliki informasi semantic bahwa tipenya adalah `Integer`.

Pada expression seperti:

```
1 a + 5
```

tipe hasil expression dapat ditentukan dari tipe operand. Jika `a` bertipe `Integer` dan `5` juga bertipe `Integer`, maka hasil expression bertipe `Integer`. Informasi seperti ini kemudian dipakai untuk type checking dan anotasi Decorated AST.

1.10 Semantic Visitor / Visitor Pattern

Semantic Visitor adalah mekanisme untuk menelusuri AST dan menjalankan pengecekan semantic pada setiap node. Setiap jenis node dapat memiliki fungsi visit sendiri, misalnya:

```
1 visitVarDecl(node)
2 visitAssign(node)
3 visitBinOp(node)
4 visitIf(node)
5 visitArrayAccess(node)
6 visitFieldAccess(node)
```

Fungsi visitor melakukan pekerjaan sesuai jenis node. Misalnya, `visitVarDecl` memasukkan identifer ke symbol table, `visitAssign` memeriksa compatibility assignment, dan `visitBinOp` menentukan tipe hasil operasi.

Traversal dilakukan menggunakan Depth-First Search (DFS), sehingga child node diproses terlebih dahulu sebelum parent node. Hal ini penting karena tipe parent node biasanya bergantung pada tipe child node.

Contoh:

```
1 x := a + 5;
```

Urutan pengecekan dapat dilakukan sebagai berikut:

```
1 1. visit VarNode(x)
2 2. visit VarNode(a)
3 3. visit NumberNode(5)
4 4. visit BinOpNode(+)
5 5. visit AssignNode
```

Setelah traversal selesai, AST akan memiliki anotasi tipe data, referensi symbol table, lexical level, dan status valid/error. Hasil akhirnya disebut Decorated AST.

2 Perancangan dan Implementasi

2.1 Bahasa dan Library yang Digunakan

Program semantic analyzer dikembangkan menggunakan bahasa C++. Bahasa ini digunakan karena program pada milestone sebelumnya juga sudah dibuat dalam C++, sehingga komponen lexer, parser, dan semantic analyzer dapat digabungkan dalam satu pipeline.

Library yang digunakan bersifat standar dan hanya dipakai untuk membantu pengelolaan data, bukan untuk membangkitkan compiler secara otomatis. Beberapa library utama yang digunakan adalah sebagai berikut:

1. `<iostream>`

Digunakan untuk proses input dan output, terutama saat menampilkan hasil semantic analysis, error message, dan struktur tree.

2. `<fstream>`

Digunakan untuk membaca file masukan dan menulis hasil keluaran seperti AST, Decorated AST, dan symbol table ke file.

3. `<string>`

Digunakan untuk menyimpan nama identifier, nama tipe data, token value, dan pesan error.

4. `<vector>`

Digunakan untuk menyimpan daftar child node pada AST, daftar parameter, daftar deklarasi, dan daftar statement.

5. `<memory>`

Digunakan untuk mengelola pointer node tree menggunakan `shared_ptr`, sehingga hubungan antar node dapat dibuat tanpa perlu mengatur memori secara manual.

6. `<unordered_map>` atau `<map>`

Digunakan untuk membantu proses lookup identifier pada symbol table.

Secara umum, library yang digunakan hanya berfungsi sebagai struktur data pendukung. Proses semantic analysis, pembentukan AST, pengelolaan symbol table, dan pengecekan tipe tetap diimplementasikan sendiri di dalam program.

2.2 Struktur Program

Program dibagi menjadi beberapa komponen agar setiap tahap compiler memiliki tanggung jawab yang jelas. Struktur ini juga mempermudah pengembangan dari milestone sebelumnya, karena semantic analyzer dapat menggunakan output dari parser.

Komponen utama program adalah sebagai berikut:

1. **Lexer**

Lexer bertugas membaca source code dan mengubahnya menjadi daftar token. Komponen ini berasal dari milestone 1. Token yang dihasilkan lexer menjadi masukan untuk parser.

2. **Parser**

Parser bertugas membaca daftar token dan membentuk parse tree. Komponen ini berasal dari milestone 2. Parse tree yang dihasilkan parser menjadi masukan untuk proses semantic analysis.

3. **AST Builder**

AST Builder bertugas mengubah parse tree menjadi Abstract Syntax Tree (AST). Pada tahap ini, node-node grammar yang tidak diperlukan untuk semantic analysis dihilangkan.

4. **Symbol Table**

Symbol table menyimpan informasi identifier, tipe data, scope, lexical level, parameter, array, record, procedure, dan function. Pada milestone ini, symbol table dibagi menjadi **tab**, **atab**, dan **btab**.

5. **Semantic Visitor**

Semantic Visitor bertugas menelusuri AST dan melakukan pengecekan semantic. Proses yang dilakukan meliputi lookup identifier, type checking, scope checking, dan pemberian anotasi pada AST.

6. **Error Handler**

Error handler menyimpan dan menampilkan error semantic yang ditemukan. Error tidak langsung menghentikan seluruh program agar semantic analyzer tetap dapat menemukan lebih dari satu error dalam satu kali eksekusi.

Alur hubungan antar komponen dapat digambarkan sebagai berikut:

```

1 Source Code
2   |
3   v
4 Lexer
5   |
6   v
7 List of Token
8   |
9   v
10 Parser
11  |
12  v
13 Parse Tree
14  |
15  v
16 AST Builder
17  |
18  v
19 AST
20  |

```

```

21      v
22 Semantic Visitor + Symbol Table
23      |
24      v
25 Decorated AST + Semantic Error Report

```

2.3 Alur Kerja Semantic Analysis

Tahap semantic analysis dimulai setelah parser berhasil membentuk parse tree. Parse tree tersebut tidak langsung digunakan untuk pengecekan semantic, karena strukturnya masih terlalu mengikuti grammar. Oleh karena itu, parse tree terlebih dahulu dikonversi menjadi AST.

Alur kerja semantic analysis pada program adalah sebagai berikut:

1. Menerima parse tree dari parser

Parser menghasilkan parse tree berdasarkan grammar bahasa Arion. Parse tree ini masih memuat node terminal dan non-terminal secara lengkap.

Contoh potongan parse tree untuk assignment:

```

1 <assignment-statement>
2 |-- <variable>
3 |   '-- ident(x)
4 |-- becomes
5 '-- <expression>
6     '-- intcon(10)

```

2. Mengubah parse tree menjadi AST

Parse tree dikonversi menjadi AST menggunakan semantic action. Informasi syntax yang tidak diperlukan, seperti token `becomes` atau node grammar perantara, tidak dimasukkan sebagai node utama.

```

1 AssignNode
2 |-- VarNode(x)
3 '-- NumberNode(10)

```

3. Inisialisasi symbol table

Sebelum traversal AST dilakukan, symbol table diisi dengan predefined identifier seperti `Integer`, `Real`, `Char`, `Boolean`, `String`, `True`, dan `False`.

Jika program mendukung fungsi bawaan seperti `readln` atau `writeln`, identifier tersebut juga dimasukkan pada tahap inisialisasi.

4. Menelusuri deklarasi

Semantic analyzer membaca bagian deklarasi untuk memasukkan identifier ke symbol table. Pada tahap ini juga dilakukan pengecekan redeklarasikan pada scope yang sama.

Contoh:

```

1 var
2     x : integer;
3     y : real;

```

menghasilkan entri symbol table untuk x dan y .

5. Menelusuri statement dan expression

Setelah deklarasi diproses, semantic analyzer menelusuri statement pada program. Setiap penggunaan identifier akan dicek melalui lookup ke symbol table.

Contoh:

```
1 x := y + 1;
```

Pada statement tersebut, semantic analyzer mencari x dan y pada symbol table, lalu memeriksa apakah tipe expression $y + 1$ compatible dengan tipe x .

6. Melakukan type checking

Type checking dilakukan pada expression, assignment, operator, procedure/function call, array access, dan record access.

Contoh error:

```
1 x := 'halo';
```

Jika x bertipe `Integer`, maka assignment tersebut menghasilkan semantic error karena `String` tidak compatible dengan `Integer`.

7. Memberi anotasi pada AST

Setelah node selesai diperiksa, node tersebut diberi informasi tambahan seperti tipe data, lexical level, referensi ke symbol table, dan status valid/error. AST yang telah diberi anotasi ini disebut Decorated AST.

Contoh:

```
1 VarNode(x) [type: integer, lev: 0, tab_ref: 33]
```

8. Menghasilkan output

Output dari semantic analysis adalah Decorated AST, symbol table, dan daftar semantic error jika ada.

Secara ringkas, alur semantic analysis dapat ditulis sebagai berikut:

1. Ambil parse tree dari parser.
2. Konversi parse tree menjadi AST.
3. Inisialisasi predefined identifier.
4. Bangun symbol table dari deklarasi.
5. Traverse AST menggunakan semantic visitor.
6. Lakukan lookup identifier dan scope checking.
7. Lakukan type checking dan compatibility checking.
8. Beri anotasi pada AST.
9. Cetak Decorated AST, symbol table, dan semantic error.

2.4 Desain Abstract Syntax Tree

AST pada program direpresentasikan menggunakan `struct` dengan base class `ASTNode`. Dari base class tersebut, node dibagi menjadi dua kelompok utama, yaitu `ExprNode` untuk expression dan `StmtNode` untuk statement.

Pembagian ini dibuat agar node expression dan statement dapat diproses secara berbeda. Expression menghasilkan nilai dan tipe data, sedangkan statement merepresentasikan aksi program.

```

1 struct ASTNode {
2     virtual ~ASTNode() = default;
3 };
4
5 struct ExprNode : public ASTNode {};
6 struct StmtNode : public ASTNode {};

```

Node expression yang digunakan pada AST adalah sebagai berikut:

1. `NumberNode`, untuk literal angka seperti 10 atau 3.14.
2. `StringNode`, untuk literal string dan character.
3. `VarNode`, untuk identifier yang digunakan sebagai variabel.
4. `UnaryOpNode`, untuk operator unary seperti `not` atau tanda minus.
5. `BinOpNode`, untuk operator biner seperti `+`, `-`, `*`, `and`, atau operator relasional.
6. `FuncCallNode`, untuk pemanggilan function yang menghasilkan nilai.

Node statement yang digunakan adalah sebagai berikut:

1. `AssignNode`, untuk assignment statement.
2. `BlockNode`, untuk kumpulan statement dalam `begin` dan `end`.
3. `IfNode`, untuk percabangan.
4. `WhileNode`, untuk perulangan `while`.
5. `ForNode`, untuk perulangan `for`.
6. `ProcCallNode`, untuk pemanggilan procedure.

Selain expression dan statement, terdapat juga node level atas seperti `VarDeclNode`, `ParamNode`, `ProcDeclNode`, `FuncDeclNode`, dan `ProgramNode`. Node-node tersebut digunakan untuk menyimpan deklarasi dan struktur utama program.

Struktur utama program disimpan pada `ProgramNode`:

```

1 struct ProgramNode : public ASTNode {
2     std::string name;
3     std::vector<std::shared_ptr<ASTNode>> declarations;
4     std::shared_ptr<BlockNode> main_block;
5 };

```

Dengan struktur tersebut, AST program terdiri dari nama program, daftar deklarasi, dan block utama.

Contoh statement:

```
1 x := a + 5;
```

dapat direpresentasikan menjadi AST berikut:

```
1 AssignNode
2 |-- VarNode(x)
3 '-- BinOpNode(+)
4   |-- VarNode(a)
5   '-- NumberNode(5)
```

Pada AST tersebut, token syntax seperti `becomes` dan `semicolon` tidak disimpan sebagai node. Informasi assignment sudah terwakili oleh `AssignNode`.

2.5 Representasi Decorated AST

Decorated AST pada program tidak menggunakan class AST yang sama persis dengan AST biasa. Untuk hasil semantic analysis, digunakan struktur `SemanticNode`. Node ini menyimpan label, daftar anotasi, dan daftar child.

```
1 struct SemanticNode
2 {
3     std::string label;
4     std::vector<std::string> annotations;
5     std::vector<std::shared_ptr<SemanticNode>> children;
6 };
```

Field `label` digunakan untuk menyimpan nama node, misalnya `Program`, `VarDecl(x)`, `Assign`, atau `BinOp(+)`. Field `annotations` digunakan untuk menyimpan informasi hasil semantic analysis, seperti:

1. `type`, yaitu tipe data node;
2. `tab_index`, yaitu referensi ke entri `tab`;
3. `block_index`, yaitu referensi ke entri `btabs`;
4. `lev`, yaitu lexical level;
5. `ref`, yaitu referensi tambahan ke `atab` atau `btabs`.

Contoh Decorated AST untuk deklarasi variabel:

```
1 VarDecl(a) -> tab_index:38, type:integer, ref:0, lev:0
```

Contoh Decorated AST untuk penggunaan variabel:

```
1 Var(a) -> tab_index:38, type:integer, lev:0
```

Jika suatu identifier tidak ditemukan, semantic analyzer tetap membentuk node, tetapi memberi anotasi `tab_index:0` dan `type:unknown`. Dengan cara ini, proses semantic analysis masih bisa dilanjutkan untuk menemukan error lain.

```
1 Var(c) -> tab_index:0, type:unknown, lev:0
```

2.6 Implementasi Syntax-Directed Translation

Konversi parse tree menjadi AST dilakukan oleh fungsi builder pada file AST. Entry point-nya adalah fungsi `buildAST`. Fungsi ini menerima `root` parse tree, lalu memanggil `buildProgram` jika `root` merupakan node `<program>`.

```

1 std::shared_ptr<ASTNode> buildAST(std::shared_ptr<ParseNode> root) {
2     if (!root) return nullptr;
3     if (root->label == "<program>") {
4         return buildProgram(root);
5     }
6     return nullptr;
7 }

```

Proses pembentukan AST dibagi menjadi beberapa fungsi sesuai jenis node parse tree:

1. `buildProgram`, untuk membentuk `ProgramNode`.
2. `buildBlock`, untuk membentuk `BlockNode`.
3. `buildStatement`, untuk membentuk node statement.
4. `buildExpression`, untuk membentuk node expression.
5. `buildSimpleExpression`, `buildTerm`, dan `buildFactor`, untuk menangani prioritas operator expression.

Pada `buildProgram`, nama program diambil dari `<program-header>`. Setelah itu, bagian deklarasi dibaca dan dikonversi menjadi node deklarasi seperti `VarDeclNode`, `ProcDeclNode`, atau `FuncDeclNode`. Block utama program dibentuk dari `compound-statement`.

Pada `buildStatement`, jenis statement ditentukan dari label parse tree. Misalnya, jika label aktual adalah `<assignment-statement>`, maka builder membuat `AssignNode`. Jika label aktual adalah `<if-statement>`, maka builder membuat `IfNode`.

Contoh aturan pada assignment:

```

1 <assignment-statement> -> <variable> becomes <expression>

```

dibentuk menjadi:

```

1 AssignNode
2 |-- VarNode(...)
3 '-- ExprNode(...)

```

Pada expression, fungsi `buildExpression`, `buildSimpleExpression`, dan `buildTerm` membentuk `BinOpNode` secara bertahap. Hal ini membuat prioritas operator tetap terjaga. Operator pada level `term`, seperti perkalian dan pembagian, diproses lebih dulu dibandingkan operator pada `simple-expression`, seperti penjumlahan dan pengurangan.

Contoh expression:

```

1 a + b * 5

```

dibentuk menjadi:

```

1 BinOpNode(+)
2 |-- VarNode(a)
3 '-- BinOpNode(*)
4    |-- VarNode(b)
5    '-- NumberNode(5)

```

Literal dan identifier dibentuk pada `buildFactor`. Token `intcon` dan `realcon` menjadi `NumberNode`, token `string` dan `charcon` menjadi `StringNode`, sedangkan token `ident` menjadi `VarNode`.

2.7 Semantic Action yang Digunakan

Semantic action pada program digunakan untuk membentuk AST dari parse tree. Aksi ini tidak selalu ditulis langsung pada parser, tetapi diterapkan pada fungsi builder seperti `buildProgram`, `buildStatement`, dan `buildExpression`.

Beberapa semantic action utama yang digunakan adalah sebagai berikut.

1. Program

```

1 <program> -> <program-header> <declaration-part>
2              <compound-statement> period

```

Semantic action:

```

1 program.node =
2   ProgramNode(program_name, declarations, main_block)

```

Pada implementasi, nama program diambil dari `program-header`, deklarasi diambil dari `declaration-part`, dan block utama diambil dari `compound-statement`.

2. Deklarasi variabel

```

1 <var-declaration> -> varsy <identifier-list> colon <type> semicolon

```

Semantic action:

```

1 for each identifier in identifier_list:
2   declaration.node = VarDeclNode(identifier, type)

```

Jika terdapat deklarasi:

```

1 var
2   a, b : integer;

```

maka builder membentuk dua node deklarasi:

```

1 VarDeclNode(a, integer)
2 VarDeclNode(b, integer)

```

3. Assignment statement

```

1 <assignment-statement> -> <variable> becomes <expression>

```

Semantic action:

```

1 assignment.node =
2   AssignNode(VarNode(variable_name), expression.node)

```

Contoh:

```
1 x := 10;
```

menjadi:

```
1 AssignNode
2 |-- VarNode(x)
3 '-- NumberNode(10)
```

4. Binary expression

```
1 <expression> -> <simple-expression> <relational-operator>
2                  <simple-expression>
```

Semantic action:

```
1 expression.node =
2   BinOpNode(operator, left.node, right.node)
```

Aksi yang sama juga dipakai untuk operator pada `simple-expression` dan `term`. Karena itu, `expression` seperti `a + b * 5` dapat tetap mempertahankan prioritas operator.

5. Unary expression

```
1 <factor> -> notsy <factor>
```

Semantic action:

```
1 factor.node = UnaryOpNode("notsy", factor.node)
```

Selain `notsy`, tanda plus dan minus pada awal `simple-expression` juga dapat dibentuk sebagai `UnaryOpNode`.

6. If statement

```
1 <if-statement> -> ifsy <expression> then <statement>
2                  (elsesy <statement>)?
```

Semantic action:

```
1 if.node =
2   IfNode(condition.node, true_branch, false_branch)
```

Pada implementasi, branch `then` dan `else` disimpan sebagai `BlockNode`. Jika branch hanya berisi satu statement, statement tersebut tetap dibungkus ke dalam `BlockNode`.

7. While statement

```
1 <while-statement> -> whilesy <expression> dosy
2                  <compound-statement> semicolon
```

Semantic action:

```
1 while.node = WhileNode(condition.node, body.node)
```

Body while diambil dari `compound-statement` dan dibentuk menjadi `BlockNode`.

8. For statement

```

1 <for-statement> -> forsy ident becomes <expression>
2                   (tosy | downtosy) <expression>
3                   dosy <compound-statement> semicolon

```

Semantic action:

```

1 for.node =
2   ForNode(initial_assignment, direction, end_expression, body)

```

Arah perulangan disimpan sebagai string "to" atau "downto".

9. Procedure dan function call

```

1 <procedure/function-call> -> ident ( <parameter-list> )?

```

Semantic action:

```

1 call.node = ProcCallNode(name, args)

```

Jika call digunakan sebagai factor dalam expression, node yang dibentuk adalah FuncCallNode. Jika call digunakan sebagai statement, node yang dibentuk adalah ProcCallNode.

2.8 Implementasi Symbol Table

Symbol table pada program disimpan di dalam `SemanticResult`. Struktur ini menyimpan hasil semantic analysis, yaitu Decorated AST, `tab`, `btabs`, `atab`, dan daftar error.

```

1 struct SemanticResult
2 {
3     std::shared_ptr<SemanticNode> ast;
4     std::vector<TabEntry> tab;
5     std::vector<BTabEntry> btabs;
6     std::vector<ATabEntry> atab;
7     std::vector<std::string> errors;
8 };

```

Tabel utama yang digunakan adalah `tab`. Setiap entri `tab` direpresentasikan oleh `TabEntry`.

```

1 struct TabEntry
2 {
3     std::string identifier;
4     int link = 0;
5     std::string obj;
6     std::string type;
7     int ref = 0;
8     int nrm = 1;
9     int lev = 0;
10    int adr = 0;
11    bool initialized = false;
12 };

```

Atribut pada `TabEntry` digunakan sebagai berikut:

1. `identifier`, menyimpan nama identifier.

2. **link**, menyimpan indeks identifier sebelumnya pada block yang sama.
3. **obj**, menyimpan jenis identifier, misalnya **variable**, **constant**, **type**, **procedure**, **function**, atau **parameter**.
4. **type**, menyimpan tipe data identifier.
5. **ref**, menyimpan referensi ke tabel lain jika tipe merupakan array, record, procedure, atau function.
6. **nrm**, menyimpan penanda parameter normal atau reference.
7. **lev**, menyimpan lexical level.
8. **adr**, menyimpan alamat atau offset.
9. **initialized**, menandai apakah identifier sudah memiliki nilai awal.

Untuk block, digunakan **BTabEntry**.

```

1 struct BTabEntry
2 {
3     int last = 0;
4     int lpar = 0;
5     int psze = 0;
6     int vsze = 0;
7 };

```

Atribut **last** menyimpan indeks identifier terakhir pada block tersebut. Atribut **lpar** menyimpan parameter terakhir, sedangkan **psze** dan **vsze** menyimpan ukuran parameter dan variabel lokal.

Untuk array, digunakan **ATabEntry**.

```

1 struct ATabEntry
2 {
3     std::string xtyp;
4     std::string etyp;
5     int eref = 0;
6     int low = 0;
7     int high = 0;
8     int elsz = 1;
9     int size = 0;
10 };

```

Atribut **xtyp** menyimpan tipe index array, **etyp** menyimpan tipe elemen array, **low** dan **high** menyimpan batas index, dan **size** menyimpan jumlah elemen array.

Sebelum semantic analysis dijalankan, symbol table diinisialisasi terlebih dahulu. Program memasukkan reserved word, predefined type, predefined constant, dan predefined procedure.

Predefined identifier yang digunakan adalah:

```

1 real
2 integer
3 char
4 boolean

```

```

5 string
6 true
7 false
8 readln
9 writeln

```

Tipe dasar seperti `integer`, `real`, `char`, `boolean`, dan `string` dimasukkan sebagai `type`. Nilai `true` dan `false` dimasukkan sebagai `constant` bertipe `boolean`. Procedure `readln` dan `writeln` dimasukkan sebagai `procedure` bertipe `void`.

Scope aktif disimpan menggunakan `display`. Setiap kali compiler masuk ke block baru, fungsi `pushBlock` membuat entri baru pada `btab`, memasukkannya ke `display`, dan menaikkan `level`. Ketika keluar dari block, fungsi `popBlock` menghapus block aktif dari `display` dan menurunkan `level`.

Lookup identifier dilakukan dalam dua bentuk:

1. `lookupCurrent`, untuk mencari identifier hanya pada scope aktif. Fungsi ini digunakan untuk mendeteksi redeklarasasi.
2. `lookup`, untuk mencari identifier dari scope terdalam ke scope terluar. Fungsi ini digunakan ketika identifier dipakai pada statement atau expression.

Dengan struktur ini, semantic analyzer dapat membedakan identifier lokal dan global, mendeteksi redeklarasasi pada scope yang sama, serta memberi anotasi `tab_index` dan `lev` pada Decorated AST.

2.9 Implementasi Scope Management

Scope pada semantic analyzer dikelola menggunakan `display`, `level`, dan `btab`. Variabel `level` menyimpan lexical level saat ini, sedangkan `display` menyimpan indeks block aktif pada setiap level.

Pada awal semantic analysis, program membuat entri awal pada `btab` dan memasukkan block global ke `display`. Setelah itu, setiap kali analyzer masuk ke block baru, fungsi `pushBlock()` dipanggil. Sebaliknya, ketika keluar dari block, fungsi `popBlock()` dipanggil.

```

1 pushBlock():
2     buat entri baru pada btab
3     masukkan indeks block ke display
4     level++
5
6 popBlock():
7     keluarkan block aktif dari display
8     level--

```

Dengan cara ini, semantic analyzer dapat mengetahui scope aktif ketika suatu deklarasi atau statement sedang diproses.

Contoh:

```

1 program ContohScope;
2 var

```

```

3      x : integer;
4
5  procedure Test;
6  var
7      y : integer;
8  begin
9      y := x + 1;
10 end;
11
12 begin
13     x := 10;
14 end.

```

Pada contoh tersebut, `x` berada pada lexical level 0, sedangkan `y` berada pada lexical level 1. Ketika analyzer berada di dalam procedure `Test`, lookup identifier dilakukan dari scope terdalam terlebih dahulu. Jika identifier tidak ditemukan pada scope lokal, pencarian dilanjutkan ke scope luar.

Pencarian identifier dilakukan dengan dua fungsi:

1. `lookupCurrent(name)`

Fungsi ini hanya mencari identifier pada scope aktif. Fungsi ini digunakan untuk mendeteksi redeklarasasi pada scope yang sama.

2. `lookup(name)`

Fungsi ini mencari identifier dari scope terdalam hingga scope terluar. Fungsi ini digunakan ketika identifier dipakai pada statement atau expression.

Contoh kasus redeklarasasi:

```

1 program SalahRedeclare;
2 var
3     x : integer;
4     x : real;
5 begin
6     x := 1;
7 end.

```

Ketika deklarasi `x` kedua diproses, semantic analyzer memanggil `lookupCurrent("x")`. Karena `x` sudah ada pada scope aktif, program menghasilkan semantic error berupa redeklarasasi.

Pada setiap entri `tab`, lexical level identifier disimpan pada atribut `lev`. Informasi ini kemudian ditampilkan juga pada Decorated AST, misalnya:

```

1 Var(x) -> tab_index:..., type:integer, lev:0

```

2.10 Implementasi Type Compatibility Rule

Type compatibility digunakan untuk menentukan apakah dua tipe data dapat digunakan bersama. Pada implementasi, pengecekan compatibility dasar dilakukan oleh fungsi `sameType()`.

Aturan dasar yang digunakan adalah:

1. Jika salah satu tipe adalah **unknown**, pengecekan dianggap lolos sementara agar error lanjutan tidak terlalu banyak.
2. Jika kedua tipe sama, maka keduanya compatible.
3. Jika tipe tujuan adalah **real** dan tipe nilai adalah **integer**, maka assignment dianggap compatible.

Dalam bentuk sederhana, aturan tersebut dapat ditulis sebagai berikut:

```

1 sameType(a, b):
2     if a == unknown or b == unknown:
3         return true
4
5     if a == b:
6         return true
7
8     if a == real and b == integer:
9         return true
10
11    return false

```

Contoh assignment valid:

```

1 program Valid;
2 var
3     x : integer;
4     y : real;
5 begin
6     x := 10;
7     y := x;
8 end.

```

Statement `y := x` valid karena `x` bertipe **Integer**, sedangkan `y` bertipe **Real**. Nilai **Integer** dapat dimasukkan ke variabel **Real**.

Contoh assignment tidak valid:

```

1 program Salah;
2 var
3     x : integer;
4     s : string;
5 begin
6     x := s;
7 end.

```

Pada statement `x := s`, tipe target adalah **Integer**, sedangkan tipe value adalah **String**. Karena kedua tipe tidak compatible, semantic analyzer menambahkan error:

```

1 assignment incompatible: integer := string

```

Untuk expression dengan operator, pengecekan tipe dilakukan melalui fungsi yang menentukan tipe hasil operasi. Operator logika seperti **and** dan **or** membutuhkan operand bertipe **Boolean**. Jika operand tidak bertipe **Boolean**, analyzer menghasilkan error.

Contoh:

```

1 program SalahLogika;
2 var

```

```

3      x : integer;
4      b : boolean;
5  begin
6      b := x and true;
7  end.

```

Expression `x and true` tidak valid karena operand kiri bertipe `Integer`. Error yang dihasilkan adalah:

```

1  logical operator requires boolean operands

```

Untuk operator aritmatika, operand `Integer` dan `Real` dianggap numerik. Jika salah satu operand bertipe `Real`, hasil expression bertipe `Real`. Jika keduanya `Integer`, hasilnya `Integer`.

Contoh:

```

1  a + b

```

Jika `a` bertipe `Integer` dan `b` bertipe `Real`, maka hasil expression bertipe `Real`.

2.11 Implementasi Type Checking

Type checking dilakukan saat semantic analyzer mengunjungi statement dan expression. Informasi tipe disimpan dalam struktur internal `TypeInfo` dan `ExprInfo`.

`TypeInfo` menyimpan nama tipe dan referensi tambahan jika tipe tersebut merupakan array atau record. Sementara itu, `ExprInfo` menyimpan tipe expression, node Decorated AST yang dibentuk, dan indeks `tab` jika expression mengacu pada identifier.

Secara umum, proses type checking dilakukan sebagai berikut:

```

1  1. Ambil tipe identifier dari symbol table.
2  2. Ambil tipe literal dari token.
3  3. Hitung tipe expression berdasarkan operator dan operand.
4  4. Bandingkan tipe target dan value pada assignment.
5  5. Tambahkan error jika tipe tidak compatible.
6  6. Beri anotasi type pada SemanticNode.

```

Contoh pada assignment:

```

1  b := a + 10;

```

Jika `x` dan `a` bertipe `Integer`, maka prosesnya:

```

1  Var(b)      -> Integer
2  Var(a)      -> Integer
3  Int(10)     -> Integer
4  a + 10      -> Integer
5  b := a + 10 -> valid

```

Decorated AST yang dihasilkan dapat berbentuk:

```

1  Assign -> type:void
2  |-- Var(b) -> tab_index:39, type:integer, lev:0
3  '-- BinOp(plus) -> type:integer
4     |-- Var(a) -> tab_index:38, type:integer, lev:0
5     '-- Int(10) -> type:integer

```

Pada assignment, jika tipe target dan value tidak compatible, error akan ditambahkan ke daftar error. Namun node tetap dibentuk agar proses semantic analysis dapat terus berjalan dan menemukan error lain.

2.12 Implementasi Semantic Visitor

Semantic analysis dilakukan oleh class `SemanticAnalyzer`. Class ini menelusuri parse tree hasil parser dan menghasilkan Decorated AST dalam bentuk `SemanticNode`. Walaupun secara konsep semantic analysis bekerja pada AST, pada implementasi ini traversal dilakukan langsung dari struktur `ParseNode` dan output-nya berupa tree semantic yang sudah terannotasi.

Entry point semantic analysis adalah fungsi:

```
1 SemanticResult analyzeSemantics(const std::shared_ptr<ParseNode> &root)
```

Fungsi tersebut membuat objek `SemanticAnalyzer`, lalu memanggil method `analyze`. Hasil akhirnya disimpan dalam `SemanticResult`.

Beberapa fungsi visitor utama yang digunakan adalah sebagai berikut:

1. visitProgram

Fungsi ini memproses root program, memasukkan nama program ke `tab`, memproses deklarasi, membuat block utama, dan memproses compound statement utama.

2. visitDeclarationPart

Fungsi ini memproses bagian deklarasi. Child node dapat berupa deklarasi konstanta, tipe, variabel, procedure, atau function.

3. visitVarDeclaration

Fungsi ini membaca daftar identifier dan tipe data, lalu memasukkan setiap variabel ke `tab`. Jika identifier sudah ada pada scope aktif, error redeklarasikan ditambahkan.

4. visitSubprogramDeclaration

Fungsi ini memproses procedure dan function. Nama subprogram dimasukkan ke `tab`, kemudian dibuat block baru untuk parameter, deklarasi lokal, dan body subprogram.

5. visitAssignment

Fungsi ini memproses sisi kiri dan kanan assignment. Setelah tipe target dan value diketahui, keduanya dibandingkan dengan aturan compatibility.

6. visitIf, visitWhile, dan visitRepeat

Fungsi-fungsi ini memeriksa apakah kondisi statement bertipe `Boolean`. Jika tidak, semantic analyzer menambahkan error.

7. visitFor

Fungsi ini memeriksa variabel counter, expression awal, expression akhir, dan body perulangan. Variabel counter harus bertipe ordinal, yaitu `Integer` atau `Char`.

8. visitCall

Fungsi ini memeriksa apakah procedure atau function yang dipanggil sudah dideklarasikan. Argumen aktual juga diproses sebagai expression.

Contoh alur visitor untuk program sederhana:

```
1 program Contoh;
2 var
3     x : integer;
4 begin
5     x := 10;
6 end.
```

Alur pemrosesan:

```
1 visitProgram
2 |-- visitDeclarationPart
3 |   '-- visitVarDeclaration
4 '-- visitCompoundStatement
5   '-- visitStatementList
6     '-- visitAssignment
7         |-- variableInfo(x)
8         '-- expressionInfo(10)
```

Hasil akhir berupa Decorated AST:

```
1 Program -> tab_index:..., lev:0
2 |-- Declarations
3 |   '-- VarDecl(x) -> tab_index:..., type:integer, ref:0, lev:0
4 '-- Compound -> block_index:..., lev:1
5   '-- Statements
6     '-- Assign -> type:void
7         |-- Var(x) -> tab_index:..., type:integer, lev:0
8         '-- Int(10) -> type:integer
```

2.13 Implementasi Error Handling Semantic

Error semantic tidak langsung menghentikan program. Setiap error disimpan ke dalam `result.errors`. Dengan cara ini, program tetap dapat melanjutkan traversal dan melaporkan lebih dari satu error dalam satu kali eksekusi.

Fungsi yang digunakan untuk mencatat error adalah:

```
1 error(message):
2     result.errors.push_back(message)
```

Beberapa error semantic yang ditangani adalah sebagai berikut:

1. Redclaration

Terjadi ketika identifier dideklarasikan lebih dari satu kali pada scope yang sama.

```
1 redeclaration in same scope: x
```

2. Unknown type

Terjadi ketika tipe yang digunakan pada deklarasi tidak ditemukan pada symbol table.

```
1 unknown type: nama_tipe
```

3. Identifier bukan type

Terjadi ketika suatu identifier digunakan sebagai tipe, tetapi entri symbol table-nya bukan type.

```
1 nama is not a type identifier
```

4. Assignment incompatible

Terjadi ketika tipe target dan value pada assignment tidak compatible.

```
1 assignment incompatible: integer := string
```

5. Kondisi bukan Boolean

Terjadi pada if, while, atau repeat jika expression kondisi tidak bertipe Boolean.

```
1 if condition must be boolean, got integer
2 while condition must be boolean, got integer
3 repeat condition must be boolean, got integer
```

6. Operator tidak sesuai tipe

Terjadi ketika operator dipakai pada operand yang tidak compatible.

```
1 logical operator requires boolean operands
2 arithmetic operands are incompatible: string and integer
```

7. Range tidak valid

Terjadi jika batas range berbeda tipe, menggunakan Real, atau batas bawah lebih besar daripada batas atas.

```
1 range bounds must have same type
2 subrange cannot use real bounds
3 range lower bound is greater than upper bound
```

Selain error, program juga dapat memberi warning pada penggunaan variabel yang belum diinisialisasi. Jika sebuah variabel digunakan sebelum diberi nilai, node tersebut diberi anotasi:

```
1 warning:possibly_uninitialized
```

Pendekatan ini membuat laporan error lebih informatif karena pengguna tidak hanya mengetahui bahwa program salah, tetapi juga mengetahui jenis kesalahan semantic yang terjadi.

2.14 Implementasi Output Semantic Analysis

Output semantic analysis dicetak melalui fungsi `printSemanticResult`. Fungsi ini menampilkan empat bagian utama:

1. Decorated AST,
2. symbol table `tab`,
3. symbol table `bt`,
4. symbol table `atab`,
5. daftar semantic error.

Bagian Decorated AST ditampilkan dalam bentuk tree. Setiap node dapat memiliki anotasi setelah tanda panah.

Contoh format:

```

1  === DECORATED AST ===
2  Program -> tab_index:37, lev:0
3  |-- Declarations
4  |   '-- VarDecls
5  |       |-- VarDecl(a) -> tab_index:38, type:integer, ref:0, lev:0
6  |       '-- VarDecl(b) -> tab_index:39, type:integer, ref:0, lev:0
7  '-- Compound -> block_index:1, lev:1
8      '-- Statements
9          |-- Assign -> type:void
10         |   |-- Var(a) -> tab_index:38, type:integer, lev:0
11         |   '-- Int(5) -> type:integer
12         '-- Assign -> type:void
13         |   |-- Var(b) -> tab_index:39, type:integer, lev:0
14         |   '-- BinOp(plus) -> type:integer
15         |       |-- Var(a) -> tab_index:38, type:integer, lev:0
16         |       '-- Int(10) -> type:integer

```

Tabel `tab` dicetak dengan kolom:

```

1  idx identifier obj type ref nrm lev adr link init

```

Tabel `bt` dicetak dengan kolom:

```

1  idx last lpar psze vsze

```

Tabel `atab` dicetak dengan kolom yang berisi informasi array, seperti tipe index, tipe elemen, batas bawah, batas atas, dan ukuran array.

Jika terdapat error, bagian error akan menampilkan daftar error yang terkumpul selama traversal. Jika tidak ada error, bagian ini dapat menunjukkan bahwa semantic analysis berhasil tanpa error.

2.15 Fungsi dan Kelas Utama

Beberapa struktur dan fungsi utama yang digunakan pada semantic analysis adalah sebagai berikut.

Nama	Fungsi
SemanticAnalyzer	Class utama yang melakukan traversal semantic analysis.
SemanticResult	Menyimpan hasil semantic analysis berupa Decorated AST, tab , btab , atab , dan daftar error.
SemanticNode	Node untuk Decorated AST. Menyimpan label, anotasi, dan child.
TabEntry	Struktur entri untuk identifier table.
BTabEntry	Struktur entri untuk block table.
ATabEntry	Struktur entri untuk array table.
analyzeSemantics()	Entry point untuk menjalankan semantic analysis dari root parse tree.
printSemanticResult()	Mencetak Decorated AST, symbol table, dan daftar error.
lookup()	Mencari identifier dari scope terdalam ke scope terluar.
lookupCurrent()	Mencari identifier hanya pada scope aktif. Digunakan untuk redeclaration checking.
pushBlock() dan popBlock()	Mengatur keluar-masuk scope atau block.
sameType()	Memeriksa compatibility dasar antara dua tipe.

Table 1: Fungsi dan struktur utama semantic analysis

2.16 Justifikasi Implementasi

Implementasi semantic analyzer dibuat modular agar mudah dihubungkan dengan milestone sebelumnya. Parser tetap bertugas menghasilkan parse tree, sedangkan semantic analyzer bertugas melakukan pengecekan makna dan menghasilkan Decorated AST.

Beberapa alasan pemilihan implementasi adalah sebagai berikut:

1. Menggunakan struktur **SemanticNode**

Decorated AST membutuhkan anotasi tambahan yang tidak ada pada **ParseNode**. Karena itu, digunakan **SemanticNode** yang memiliki field **annotations**. Dengan struktur ini, informasi seperti **type**, **tab_index**, **lev**, dan **ref** dapat ditambahkan tanpa mengubah struktur parse tree.

2. Menggunakan **tab**, **btab**, dan **atab**

Struktur ini mengikuti spesifikasi milestone 3. **tab** digunakan untuk identifier, **btab** untuk block, dan **atab** untuk array. Pemisahan ini membuat informasi identifier dan tipe kompleks lebih mudah dikelola.

3. Menggunakan **display** untuk scope

`display` memudahkan proses lookup identifier dari scope terdalam ke scope terluar. Selain itu, struktur ini juga memudahkan pengelolaan lexical level saat masuk dan keluar block.

4. Error dikumpulkan, bukan langsung menghentikan program

Dengan menyimpan error pada `result.errors`, semantic analyzer dapat melaporkan beberapa error sekaligus. Hal ini lebih membantu saat debugging dibandingkan berhenti pada error pertama.

5. Output dibuat eksplisit

Program tidak hanya menampilkan apakah semantic analysis berhasil atau gagal, tetapi juga menampilkan Decorated AST dan symbol table. Output ini memudahkan pengecekan apakah deklarasi, scope, dan type checking sudah berjalan sesuai harapan.

3 Type Compatibility Rule

Type compatibility rule adalah aturan yang digunakan semantic analyzer untuk menentukan apakah suatu tipe data dapat digunakan pada konteks tertentu. Aturan ini dipakai pada assignment, expression, operator, subrange, array, record, scope, serta pemanggilan procedure dan function.

Pada implementasi, informasi tipe disimpan sebagai string, misalnya `integer`, `real`, `boolean`, `char`, `string`, `array`, dan `record`. Jika suatu tipe tidak dapat ditentukan, semantic analyzer menggunakan tipe `unknown` agar proses analisis tetap dapat dilanjutkan dan tidak berhenti pada satu error saja.

3.1 Primitive Type Rule

Primitive type adalah tipe dasar yang dapat langsung digunakan pada deklarasi, constant, dan expression. Bahasa Arion menggunakan beberapa tipe dasar, yaitu `Integer`, `Real`, `Boolean`, `Char`, dan `String`.

3.1.1 Integer

Tipe `Integer` digunakan untuk bilangan bulat. Constant yang berasal dari token `intcon` diberi tipe `integer`.

Contoh:

```
1 var
2   x : integer;
3 begin
4   x := 10;
5 end.
```

Pada contoh tersebut, literal 10 bertipe `integer`. Assignment valid karena variabel `x` juga bertipe `integer`.

3.1.2 Real

Tipe **Real** digunakan untuk bilangan riil. Constant yang berasal dari token **realcon** diberi tipe **real**.

Contoh:

```
1 var
2   r : real;
3 begin
4   r := 3.14;
5 end.
```

Assignment tersebut valid karena literal 3.14 bertipe **real** dan variabel **r** juga bertipe **real**.

Tipe **integer** juga dapat digunakan pada konteks **real**. Karena itu, assignment berikut valid:

```
1 var
2   x : integer;
3   r : real;
4 begin
5   x := 10;
6   r := x;
7 end.
```

Namun assignment dari **real** ke **integer** tidak dianggap valid tanpa konversi eksplisit.

```
1 var
2   x : integer;
3   r : real;
4 begin
5   r := 3.14;
6   x := r;
7 end.
```

Error:

```
1 assignment incompatible: integer := real
```

3.1.3 Boolean

Tipe **Boolean** digunakan untuk nilai kebenaran. Predefined constant **true** dan **false** dimasukkan ke symbol table sebagai constant bertipe **boolean**.

Contoh valid:

```
1 var
2   b : boolean;
3 begin
4   b := true;
5 end.
```

Tipe **boolean** tidak compatible dengan **integer**, **real**, **char**, maupun **string**.

Contoh tidak valid:

```
1 var
2   b : boolean;
3 begin
4   b := 1;
5 end.
```

Error:

```
1 assignment incompatible: boolean := integer
```

3.1.4 Char

Tipe **Char** digunakan untuk karakter tunggal. Constant yang berasal dari token **charcon** diberi tipe **char**.

Contoh valid:

```
1 var
2   c : char;
3 begin
4   c := 'A';
5 end.
```

Tipe **char** tidak otomatis compatible dengan **string**, walaupun keduanya ditulis menggunakan tanda petik.

3.1.5 String

Tipe **String** digunakan untuk literal string. Constant yang berasal dari token **string** diberi tipe **string**.

Contoh valid:

```
1 var
2   nama : string;
3 begin
4   nama := 'Arion';
5 end.
```

Contoh tidak valid:

```
1 var
2   x : integer;
3 begin
4   x := 'Arion';
5 end.
```

Error:

```
1 assignment incompatible: integer := string
```

Pada implementasi ini, panjang string tidak dibedakan secara eksplisit. Karena itu, compatibility string hanya dicek berdasarkan nama tipenya, yaitu **string**.

3.2 Structured Type Rule

Structured type adalah tipe yang memiliki struktur internal. Pada milestone ini, structured type yang diperiksa adalah array, record, enumerated, dan subrange.

3.2.1 Array

Array memiliki bentuk umum sebagai berikut:

```
1 array [T1] of T2
```

T1 adalah tipe index, sedangkan T2 adalah tipe elemen. Aturan array yang digunakan adalah:

1. tipe index tidak boleh **real**;
2. tipe index dapat berasal dari range atau identifier type;
3. tipe elemen dapat berupa tipe apapun yang valid;
4. informasi array disimpan pada **atab**.

Contoh valid:

```
1 var
2   data : array [1..5] of integer;
3 begin
4   data[2] := 10;
5 end.
```

Pada contoh tersebut, index array memiliki range 1..5 dengan tipe dasar **integer**. Elemen array bertipe **integer**.

Contoh tidak valid:

```
1 var
2   data : array [1.5..5.5] of integer;
3 begin
4 end.
```

Error:

```
1 array index type cannot be real
```

Informasi array disimpan pada **atab** dengan atribut seperti **xtyp**, **etyp**, **low**, **high**, dan **size**.

3.2.2 Record

Record digunakan untuk menyimpan beberapa field dalam satu tipe. Setiap field memiliki nama dan tipe masing-masing.

Contoh:

```

1 type
2     Mahasiswa == record
3         nim : integer;
4         nama : string;
5     end;
6
7 var
8     m : Mahasiswa;
9 begin
10    m.nim := 13524053;
11    m.nama := 'Budi';
12 end.

```

Aturan record yang digunakan adalah:

1. setiap field harus dideklarasikan di dalam record;
2. field dapat memiliki tipe apapun yang valid;
3. field disimpan sebagai identifier dengan `obj = field`;
4. informasi block record disimpan pada `btabs`.

Jika field yang diakses tidak ada, semantic analyzer menghasilkan error.

Contoh tidak valid:

```

1 m.umur := 20;

```

Error:

```

1 field not found: umur

```

3.2.3 Enumerated

Enumerated type adalah tipe yang nilainya berasal dari daftar identifier. Pada rancangan aturan semantic, identifier di dalam enumerated harus berada pada tipe enumerasi yang sama.

Contoh:

```

1 type
2     Warna == (Merah, Kuning, Hijau);

```

Pada implementasi saat ini, enumerated disederhanakan menjadi tipe `integer`. Artinya, semantic analyzer tetap mengenali enumerated sebagai tipe yang valid, tetapi representasi internalnya belum dibedakan sebagai tipe enumerasi tersendiri.

```

1 <enumerated> -> type integer

```

Batasan ini dicatat karena implementasi belum menyimpan daftar nilai enumerated secara eksplisit pada symbol table.

3.2.4 Subrange

Subrange adalah tipe yang memiliki batas bawah dan batas atas.

Contoh:

```
1 var
2   index : 1..10;
```

Aturan subrange yang digunakan adalah:

1. batas bawah dan batas atas harus memiliki tipe yang sama;
2. tipe batas tidak boleh **real**;
3. batas bawah tidak boleh lebih besar dari batas atas.

Contoh tidak valid karena tipe batas adalah **real**:

```
1 var
2   r : 1.5..5.5;
3 begin
4 end.
```

Error:

```
1 subrange cannot use real bounds
```

Contoh tidak valid karena batas bawah lebih besar:

```
1 var
2   x : 10..1;
3 begin
4 end.
```

Error:

```
1 range lower bound is greater than upper bound
```

3.3 Assignment Compatibility

Assignment compatibility menentukan apakah nilai di sisi kanan assignment dapat dimasukkan ke variabel di sisi kiri.

Bentuk umum assignment adalah:

```
1 target := value;
```

Aturan assignment yang digunakan adalah:

1. jika tipe target dan value sama, assignment valid;
2. jika target bertipe **real** dan value bertipe **integer**, assignment valid;
3. jika salah satu tipe **unknown**, pengecekan tidak langsung menghasilkan error tambahan;

4. selain kondisi tersebut, assignment tidak valid.

Contoh valid karena tipe sama:

```
1 var
2   x : integer;
3 begin
4   x := 10;
5 end.
```

Contoh valid karena `integer` dapat masuk ke `real`:

```
1 var
2   x : integer;
3   r : real;
4 begin
5   x := 10;
6   r := x;
7 end.
```

Contoh tidak valid:

```
1 var
2   x : integer;
3   s : string;
4 begin
5   x := s;
6 end.
```

Error:

```
1 assignment incompatible: integer := string
```

Contoh tidak valid lainnya:

```
1 var
2   x : integer;
3   r : real;
4 begin
5   r := 3.14;
6   x := r;
7 end.
```

Error:

```
1 assignment incompatible: integer := real
```

3.4 Operator Compatibility

Operator compatibility menentukan tipe operand yang boleh digunakan pada operator tertentu dan tipe hasil expression setelah operator tersebut diproses.

3.4.1 Arithmetic Operator

Operator aritmatika yang digunakan adalah `plus`, `minus`, `times`, `idiv`, `rdiv`, dan `imod`.

Aturan umum operator aritmatika:

1. operand harus bertipe numerik, yaitu `integer` atau `real`;
2. jika salah satu operand bertipe `real`, hasil operasi bertipe `real`;
3. jika kedua operand bertipe `integer`, hasil operasi bertipe `integer`;
4. operator `rdiv` selalu menghasilkan `real`.

Contoh:

```

1 var
2   a : integer;
3   b : real;
4   c : real;
5 begin
6   c := a + b;
7 end.
```

Expression `a + b` menghasilkan tipe `real`, karena salah satu operand bertipe `real`.

Contoh tidak valid:

```

1 var
2   x : integer;
3   s : string;
4 begin
5   x := s + 1;
6 end.
```

Error:

```

1 arithmetic operands are incompatible: string and integer
```

3.4.2 Relational Operator

Operator relasional digunakan untuk membandingkan dua expression. Operator yang termasuk relational operator adalah `eql`, `neq`, `gtr`, `geq`, `lss`, dan `leq`.

Aturan relational operator:

1. kedua operand harus compatible untuk dibandingkan;
2. hasil relational operator selalu bertipe `boolean`.

Contoh valid:

```

1 var
2   x : integer;
3   y : integer;
4   b : boolean;
5 begin
6   b := x < y;
7 end.
```

Expression `x < y` menghasilkan tipe `boolean`.

Contoh tidak valid:

```

1 var
2   x : integer;
3   s : string;
4   b : boolean;
5 begin
6   b := x < s;
7 end.

```

Pada contoh tersebut, `integer` dan `string` tidak compatible untuk operator relasional.

3.4.3 Logical Operator

Operator logika yang digunakan adalah `andsy`, `orsy`, dan `notsy`.

Aturan logical operator:

1. `andsy` membutuhkan operand kiri dan kanan bertipe `boolean`;
2. `orsy` membutuhkan operand kiri dan kanan bertipe `boolean`;
3. `notsy` membutuhkan satu operand bertipe `boolean`;
4. hasil operator logika selalu bertipe `boolean`.

Contoh valid:

```

1 var
2   a : boolean;
3   b : boolean;
4   c : boolean;
5 begin
6   c := a and b;
7 end.

```

Contoh tidak valid:

```

1 var
2   x : integer;
3   b : boolean;
4 begin
5   b := x and true;
6 end.

```

Error:

```

1 logical operator requires boolean operands

```

Contoh tidak valid untuk not:

```

1 var
2   x : integer;
3   b : boolean;
4 begin
5   b := not x;
6 end.

```

Error:

```

1 not operator requires boolean operand

```

3.5 Scope Rule

Scope rule menentukan identifier mana yang boleh digunakan pada suatu posisi program. Walaupun scope bukan tipe data, aturan ini tetap berpengaruh pada type checking, karena tipe identifier hanya dapat diketahui jika identifier tersebut berhasil ditemukan pada symbol table.

Aturan scope yang digunakan adalah:

1. identifier harus dideklarasikan sebelum digunakan;
2. identifier tidak boleh dideklarasikan ulang pada scope yang sama;
3. identifier pada scope lokal boleh memiliki nama yang sama dengan identifier pada scope luar;
4. lookup identifier dilakukan dari scope terdalam ke scope terluar.

Contoh valid:

```
1 program ValidScope;
2 var
3     x : integer;
4
5 procedure Test;
6 var
7     x : real;
8 begin
9     x := 3.14;
10 end;
11
12 begin
13     x := 10;
14 end.
```

Program tersebut valid karena `x` global dan `x` lokal berada pada scope berbeda.

Contoh tidak valid:

```
1 program SalahScope;
2 var
3     x : integer;
4     x : real;
5 begin
6     x := 1;
7 end.
```

Error:

```
1 redeclaration in same scope: x
```

Contoh identifier tidak ditemukan:

```
1 program Undeclared;
2 begin
3     y := 10;
4 end.
```

Error:

```
1 undeclared identifier: y
```

3.6 Function dan Procedure Rule

Procedure dan function memiliki aturan semantic yang berhubungan dengan deklarasi, parameter, dan pemanggilan.

Aturan yang digunakan adalah:

1. procedure dan function harus dideklarasikan sebelum dipanggil;
2. nama procedure atau function disimpan pada **tab**;
3. parameter formal disimpan sebagai **parameter** pada scope subprogram;
4. setiap argumen aktual diproses sebagai expression;
5. tipe argumen aktual seharusnya compatible dengan tipe parameter formal.

Contoh procedure:

```
1 program ProcValid;
2
3 procedure Cetak(x : integer);
4 begin
5 end;
6
7 begin
8     Cetak(10);
9 end.
```

Pada contoh tersebut, **Cetak** dideklarasikan sebagai procedure dan parameter **x** bertipe **integer**. Pemanggilan **Cetak(10)** valid karena argumen aktual 10 juga bertipe **integer**.

Contoh function:

```
1 program FuncValid;
2
3 function Tambah(a : integer; b : integer) : integer;
4 begin
5     Tambah := a + b;
6 end;
7
8 begin
9     Tambah(1, 2);
10 end.
```

Function memiliki return type yang disimpan pada entri function di **tab**. Jika function digunakan dalam expression, tipe expression seharusnya mengikuti return type function.

Pada implementasi saat ini, pemanggilan procedure/function sudah dicek keberadaan identifier-nya. Argumen aktual juga diproses sebagai expression. Namun pengecekan penuh jumlah parameter dan compatibility argumen terhadap parameter formal masih menjadi batasan implementasi.

Jika procedure atau function belum dideklarasikan, error yang dihasilkan adalah:

```
1 undeclared procedure/function: nama
```

3.7 Ringkasan Type Compatibility Rule

Ringkasan aturan type compatibility yang digunakan adalah sebagai berikut:

Konteks	Aturan
Primitive type	<code>intcon</code> bertipe <code>integer</code> , <code>realcon</code> bertipe <code>real</code> , <code>charcon</code> bertipe <code>char</code> , <code>string</code> bertipe <code>string</code> , dan <code>true/false</code> bertipe <code>boolean</code> .
Assignment	Tipe yang sama compatible. <code>integer</code> dapat di-assign ke <code>real</code> . Selain itu tidak compatible, kecuali salah satu tipe <code>unknown</code> .
Arithmetic operator	Operand harus numerik. Jika salah satu operand <code>real</code> , hasilnya <code>real</code> . Jika keduanya <code>integer</code> , hasilnya <code>integer</code> . Operator <code>rdiv</code> menghasilkan <code>real</code> .
Relational operator	Operand harus compatible untuk dibandingkan. Hasil operator relasional bertipe <code>boolean</code> .
Logical operator	<code>andsy</code> , <code>orsy</code> , dan <code>notsy</code> membutuhkan operand <code>boolean</code> dan menghasilkan <code>boolean</code> .
Array	Index array tidak boleh <code>real</code> . Tipe index dan tipe elemen disimpan pada <code>atab</code> .
Record	Field record disimpan pada block record. Field harus ada sebelum dapat diakses.
Subrange	Batas bawah dan batas atas harus memiliki tipe yang sama, tidak boleh <code>real</code> , dan batas bawah tidak boleh lebih besar dari batas atas.
Enumerated	Enumerated dikenali sebagai tipe valid, tetapi pada implementasi saat ini direpresentasikan sebagai <code>integer</code> .
Scope	Identifier dicari dari scope terdalam ke scope terluar. Redclaration pada scope yang sama tidak diperbolehkan.
Procedure/function	Procedure/function harus dideklarasikan sebelum dipanggil. Argumen aktual diproses sebagai expression.

Table 2: Ringkasan Type Compatibility Rule

4 Hasil Implementasi

Bagian ini menjelaskan hasil implementasi semantic analyzer pada program. Output utama yang dihasilkan adalah AST, Decorated AST, symbol table, dan daftar semantic error. Hasil ini digunakan untuk memeriksa apakah semantic analyzer sudah dapat

melakukan konversi parse tree, lookup identifier, scope checking, type checking, dan error detection.

Contoh program yang digunakan untuk menunjukkan hasil implementasi adalah sebagai berikut:

```

1 program ContohValid;
2 var
3     x : integer;
4     y : integer;
5 begin
6     x := 10;
7     y := x + 5;
8 end.
```

Program tersebut digunakan karena memiliki deklarasi variabel, assignment, penggunaan identifier, dan operasi aritmatika sederhana.

4.1 Hasil Konversi Parse Tree ke AST

Parser menghasilkan parse tree yang masih mengikuti struktur grammar. Sebagai contoh, statement:

```

1 y := x + 5;
```

pada parse tree masih memiliki node grammar seperti <assignment-statement>, <variable>, <expression>, <simple-expression>, <term>, dan <factor>.

Potongan parse tree-nya dapat digambarkan sebagai berikut:

```

1 <assignment-statement>
2 |-- <variable>
3 |   '-- ident(y)
4 |-- becomes
5 '-- <expression>
6     |-- <simple-expression>
7     |   |-- <term>
8     |   |   '-- <factor>
9     |   |   '-- ident(x)
10    |   |-- plus
11    |   '-- <term>
12    |       '-- <factor>
13    |           '-- intcon(5)
```

Setelah diproses oleh AST builder, node grammar yang tidak diperlukan dihilangkan. Hasil AST menjadi lebih ringkas:

```

1 AssignNode
2 |-- VarNode(y)
3 '-- BinOpNode(plus)
4     |-- VarNode(x)
5     '-- NumberNode(5)
```

Dari hasil tersebut terlihat bahwa AST hanya menyimpan struktur yang berpengaruh terhadap makna program, yaitu assignment, target assignment, operator, dan operand.

4.2 Hasil Decorated AST

Setelah AST diproses oleh semantic analyzer, setiap node diberi anotasi semantic. Anotasi yang ditambahkan antara lain tipe data, indeks symbol table, lexical level, dan status hasil pengecekan.

Untuk program uji:

```

1 program Hello;
2 var
3     a, b : integer;
4 begin
5     a := 5;
6     b := a + 10;
7 end.
```

Decorated AST yang dihasilkan dapat berbentuk sebagai berikut:

```

1 Program -> tab_index:37, lev:0
2 |-- Declarations
3 |   '-- VarDecls
4 |       |-- VarDecl(a) -> tab_index:38, type:integer, ref:0, lev:0
5 |       '-- VarDecl(b) -> tab_index:39, type:integer, ref:0, lev:0
6 '-- Compound -> block_index:1, lev:1
7   '-- Statements
8       |-- Assign -> type:void
9       |   |-- Var(a) -> tab_index:38, type:integer, lev:0
10      |   '-- Int(5) -> type:integer
11      '-- Assign -> type:void
12      |   |-- Var(b) -> tab_index:39, type:integer, lev:0
13      |   '-- BinOp(plus) -> type:integer
14      |       |-- Var(a) -> tab_index:38, type:integer, lev:0
15      |       '-- Int(10) -> type:integer
```

Pada hasil tersebut, program Hello disimpan pada tab indeks 37. Variabel a dan b masing-masing disimpan pada indeks 38 dan 39. Operasi $a + 10$ menghasilkan tipe `integer`, sehingga assignment ke b valid.

4.3 Hasil Symbol Table

Semantic analyzer menghasilkan tiga tabel utama, yaitu `tab`, `btab`, dan `atab`. Tabel `tab` menyimpan identifier, `btab` menyimpan informasi block, dan `atab` menyimpan informasi array.

4.3.1 Contoh Tab

Tabel `tab` menyimpan seluruh identifier yang dikenali oleh semantic analyzer. Isi tabel tidak hanya mencakup identifier dari program pengguna, tetapi juga sentinel, reserved word, predefined type, predefined constant, dan predefined procedure.

Berikut adalah potongan hasil `tab` dari program uji:

```

1 === SYMBOL TABLE: tab ===
2 idx  identifier  obj      type      ref  nrm  lev  adr  link  init
3 -----
```

4	0	<none>	sentinel	none	0	1	0	0	0	yes
5	1	and	reserved	none	0	1	0	0	0	yes
6	2	array	reserved	none	0	1	0	0	0	yes
7	...									
8	27	while	reserved	none	0	1	0	0	0	yes
9	28	real	type	real	0	1	0	0	0	yes
10	29	integer	type	integer	0	1	0	0	28	yes
11	30	char	type	char	0	1	0	0	29	yes
12	31	boolean	type	boolean	0	1	0	0	30	yes
13	32	string	type	string	0	1	0	0	31	yes
14	33	true	constant	boolean	0	1	0	1	32	yes
15	34	false	constant	boolean	0	1	0	0	33	yes
16	35	readln	procedure	void	0	1	0	0	34	yes
17	36	writeln	procedure	void	0	1	0	0	35	yes
18	37	hello	program	none	0	1	0	0	36	yes
19	38	a	variable	integer	0	1	0	0	37	yes
20	39	b	variable	integer	0	1	0	1	38	yes

Dari hasil tersebut terlihat bahwa indeks 1 sampai 27 digunakan untuk reserved word. Indeks 28 sampai 36 digunakan untuk predefined identifier, yaitu `real`, `integer`, `char`, `boolean`, `string`, `true`, `false`, `readln`, dan `writeln`.

Identifier dari program pengguna mulai masuk setelah predefined identifier. Pada contoh tersebut, `hello` disimpan sebagai `program`, sedangkan `a` dan `b` disimpan sebagai `variable` bertipe `integer`. Kolom `lev` bernilai 0 karena ketiganya berada pada scope global. Kolom `adr` pada `a` dan `b` menunjukkan alamat relatif variabel.

4.3.2 Contoh Btab

Tabel `btab` menyimpan informasi block. Potongan hasilnya dapat ditulis sebagai berikut:

1	btab
2	idx last lpar psze vsze
3	-----
4	0 34 0 0 2
5	1 0 0 0 0

Atribut `last` menunjukkan entri terakhir pada block tersebut. Atribut `vsze` menunjukkan jumlah variabel lokal yang dideklarasikan pada block.

4.3.3 Contoh Atab

Jika program tidak menggunakan array, maka `atab` tidak memiliki entri tambahan. Untuk menguji `atab`, digunakan program berikut:

```

1 program ContohArray;
2 var
3   data : array [1..5] of integer;
4 begin
5   data[2] := 10;
6 end.
```

Potongan `atab` yang dihasilkan dapat berbentuk:

```

1 atab
```

idx	xtyp	etyp	eref	low	high	elsz	size
0	integer	integer	0	1	5	1	5

Dari tabel tersebut terlihat bahwa array `data` memiliki tipe index `integer`, tipe elemen `integer`, batas bawah 1, batas atas 5, dan ukuran 5 elemen.

4.4 Hasil Type Checking

Type checking dilakukan pada assignment, expression, operator, dan kondisi statement. Untuk program:

```

1 program TypeValid;
2 var
3     x : integer;
4     y : integer;
5 begin
6     x := 10;
7     y := x + 5;
8 end.
```

hasil pengecekan tipe dapat dijelaskan sebagai berikut:

```

1 x      -> integer
2 10     -> integer
3 x := 10 -> valid
4
5 x      -> integer
6 5      -> integer
7 x + 5  -> integer
8 y      -> integer
9 y := x + 5 -> valid
```

Karena semua assignment memiliki tipe yang compatible, semantic analyzer tidak menghasilkan error.

Contoh lain untuk assignment `integer` ke `real`:

```

1 program TypeReal;
2 var
3     x : integer;
4     r : real;
5 begin
6     x := 10;
7     r := x;
8 end.
```

Assignment `r := x` valid karena aturan compatibility mengizinkan nilai `integer` dimasukkan ke variabel bertipe `real`.

4.5 Hasil Scope Checking

Scope checking dilakukan dengan mencari identifier dari scope terdalam ke scope terluar. Contoh program valid:

```

1 program ScopeValid;
2 var
3     x : integer;
4
5 procedure Test;
6 var
7     y : integer;
8 begin
9     y := x + 1;
10 end;
11
12 begin
13     x := 10;
14 end.

```

Pada program tersebut, x dideklarasikan pada scope global, sedangkan y dideklarasikan pada scope procedure **Test**. Ketika semantic analyzer memproses statement $y := x + 1$, variabel y ditemukan pada scope lokal procedure, sedangkan x ditemukan pada scope global.

Potongan Decorated AST dapat berbentuk:

```

1 ProcedureDecl(Test) -> tab_index:39, block_index:1, lev:0, type:void
2 '-- Block -> block_index:1, lev:1
3   |-- Declarations
4   |   '-- VarDecls
5   |       '-- VarDecl(y) -> tab_index:40, type:integer, ref:0, lev:1
6   '-- Compound
7   '-- Statements
8   '-- Assign -> type:void
9       |-- Var(y) -> tab_index:40, type:integer, lev:1
10      '-- BinOp(plus) -> type:integer
11          |-- Var(x) -> tab_index:38, type:integer, lev:0
12          '-- Int(1) -> type:integer

```

Hasil tersebut menunjukkan bahwa semantic analyzer dapat membedakan identifier berdasarkan lexical level.

4.6 Hasil Semantic Error Detection

Semantic analyzer juga diuji dengan program yang memiliki error semantic. Error yang ditemukan disimpan pada daftar error dan ditampilkan setelah Decorated AST dan symbol table.

4.6.1 Undeclared Identifier

Contoh program:

```

1 program UndeclaredTest;
2 begin
3     x := 10;
4 end.

```

Variabel x belum pernah dideklarasikan. Error yang dihasilkan:

```
1 undeclared identifier: x
```

Node x tetap ditampilkan pada Decorated AST, tetapi diberi tipe **unknown** agar traversal dapat dilanjutkan.

```
1 Var(x) -> tab_index:0, type:unknown, lev:1
```

4.6.2 Redclaration

Contoh program:

```
1 program RedeclareTest;
2 var
3     x : integer;
4     x : real;
5 begin
6     x := 10;
7 end.
```

Error yang dihasilkan:

```
1 redeclaration in same scope: x
```

Error ini muncul karena x dideklarasikan dua kali pada scope yang sama.

4.6.3 Type Mismatch

Contoh program:

```
1 program TypeMismatchTest;
2 var
3     x : integer;
4     s : string;
5 begin
6     x := s;
7 end.
```

Error yang dihasilkan:

```
1 assignment incompatible: integer := string
```

Error tersebut menunjukkan bahwa semantic analyzer berhasil membandingkan tipe target assignment dan tipe value.

4.6.4 Invalid If Condition

Contoh program:

```
1 program IfErrorTest;
2 var
3     x : integer;
4 begin
5     if x then
6         x := 1;
7 end.
```

Error yang dihasilkan:

```
1 if condition must be boolean, got integer
```

Kondisi if harus bertipe `boolean`, sedangkan `x` bertipe `integer`.

4.6.5 Invalid Range

Contoh program:

```
1 program RangeErrorTest;  
2 var  
3     x : 10..1;  
4 begin  
5 end.
```

Error yang dihasilkan:

```
1 range lower bound is greater than upper bound
```

Error tersebut menunjukkan bahwa semantic analyzer berhasil memeriksa batas bawah dan batas atas subrange.

4.7 Ringkasan Hasil Implementasi

Berdasarkan hasil implementasi, semantic analyzer sudah dapat menghasilkan:

1. AST hasil penyederhanaan parse tree;
2. Decorated AST dengan anotasi tipe, lexical level, dan referensi symbol table;
3. symbol table berupa `tab`, `btabs`, dan `atab`;
4. hasil type checking pada assignment dan expression;
5. hasil scope checking melalui lookup identifier;
6. daftar semantic error untuk program yang tidak valid.

Dengan hasil tersebut, semantic analyzer sudah memenuhi tujuan utama milestone 3, yaitu memeriksa kebenaran semantic program dan menghasilkan struktur antara berupa Decorated AST serta symbol table.

5 Pengujian

Pengujian dilakukan untuk memastikan semantic analyzer dapat menghasilkan Decorated AST, symbol table, dan semantic error sesuai dengan masukan program. Kasus uji dibuat untuk mencakup program valid, type checking, scope checking, penggunaan array, serta deteksi error semantic.

5.1 Ringkasan Test Case

Pengujian dilakukan menggunakan sembilan test case yang mewakili program valid, penggunaan scope, type checking, array, record, procedure, dan semantic error. Beberapa test case ditampilkan secara rinci pada laporan, sedangkan test case lainnya diringkas pada tabel berikut.

No	Test Case	Fokus Pengujian
1	Program Valid Sederhana	Deklarasi variabel, assignment, expression, dan predefined procedure
2	Type Mismatch	Assignment dengan tipe yang tidak compatible
3	Scope Valid	Lookup identifier pada scope global dan lokal
4	Redeclaration	Deklarasi ulang identifier pada scope yang sama
5	Array dan Indexing	Deklarasi array, akses elemen, dan pencatatan pada <code>atab</code>
6	Invalid If Condition	Kondisi <code>if</code> yang tidak bertipe <code>boolean</code>
7	Record dan Akses Field	Deklarasi record, field, dan akses field valid
8	Invalid Record Field	Akses field record yang tidak dideklarasikan
9	Procedure dengan Parameter	Procedure, parameter formal, scope, dan argument checking

Table 3: Ringkasan test case semantic analyzer

Tidak seluruh test case dijelaskan secara rinci pada laporan ini agar pembahasan tetap ringkas dan fokus pada fitur utama semantic analyzer. Laporan hanya menampilkan beberapa test case yang mewakili aspek penting, yaitu program valid sederhana, redeclaration, array, record, dan procedure dengan parameter.

Test case lainnya tetap digunakan dalam proses pengujian, tetapi tidak dibahas secara detail pada laporan. Seluruh input dan output lengkap untuk semua test case dapat dilihat pada repository, khususnya pada direktori `test/milestone-3/`.

5.2 Test Case 1: Program Valid Sederhana

5.2.1 Tujuan Pengujian

Menguji deklarasi variabel sederhana, assignment, operasi aritmatika, pemanggilan predefined procedure `writeln`, serta pembentukan Decorated AST dan symbol table dasar.

5.2.2 Input Program

```
1 program Hello;
2
```

```

3  var
4    a, b : integer;
5
6  begin
7    a := 5;
8    b := a + 10;
9    writeln('Result_ = ', b);
10 end.

```

5.2.3 Output Program

Output program menunjukkan bahwa tokenisasi, parsing, pembentukan Raw AST, Decorated AST, dan semantic analysis berhasil dilakukan. Bagian penting dari Decorated AST ditunjukkan sebagai berikut.

```

1  ProgramNode(name: 'Hello') -> tab_index:37, lev:0
2  Declarations
3    VarDecls
4      VarDecl(a) -> tab_index:38, type:integer, ref:0, lev:0
5      VarDecl(b) -> tab_index:39, type:integer, ref:0, lev:0
6
7  Compound -> block_index:1, lev:1
8    Statements
9      Assign -> type:void
10     Target
11       Var(a) -> tab_index:38, type:integer, lev:0
12     Value
13       Int(5) -> type:integer
14
15     Assign -> type:void
16     Target
17       Var(b) -> tab_index:39, type:integer, lev:0
18     Value
19       BinOp(plus) -> type:integer
20       Var(a) -> tab_index:38, type:integer, lev:0
21       Int(10) -> type:integer
22
23     Call(writeln) -> tab_index:36, lev:1
24     Args
25       String('Result_ = ') -> type:string, len:9
26       Var(b) -> tab_index:39, type:integer, lev:0

```

Bagian symbol table yang relevan adalah sebagai berikut.

idx	identifier	obj	type	ref	lev	adr	init
36	writeln	procedure	void	0	0	0	yes
37	hello	program	void	0	0	0	yes
38	a	variable	integer	0	0	0	yes
39	b	variable	integer	0	0	1	yes

Bagian btab yang relevan adalah sebagai berikut.

idx	last	lpar	psze	vsze
0	39	0	0	2
1	0	0	0	0

Semantic diagnostics yang dihasilkan adalah sebagai berikut.

```
1 No semantic errors.
```

5.2.4 Hasil Pengujian

Semantic analyzer berhasil:

1. memasukkan program `hello` ke dalam `tab`;
2. memasukkan variabel `a` dan `b` sebagai `variable` bertipe `integer`;
3. memberi anotasi `tab_index`, `type`, dan `lev` pada deklarasi serta penggunaan variabel;
4. menentukan bahwa assignment `a := 5` valid;
5. menentukan bahwa expression `a + 10` bertipe `integer`;
6. mengenali `writeln` sebagai predefined procedure;
7. menghitung panjang string `'Result = '` sebagai `len:9`;
8. menyelesaikan semantic analysis tanpa error.

Berdasarkan hasil tersebut, Test Case 1 dinyatakan **berhasil**.

5.3 Test Case 4: Redeclaration pada Scope yang Sama

5.3.1 Tujuan Pengujian

Menguji kemampuan semantic analyzer dalam mendeteksi deklarasi ulang identifier pada scope yang sama. Pada test case ini, variabel `x` dideklarasikan dua kali pada block yang sama dengan tipe yang berbeda.

5.3.2 Input Program

```
1 program RedeclareTest;  
2  
3 var  
4     x : integer;  
5     x : real;  
6  
7 begin  
8     x := 10;  
9 end.
```

5.3.3 Output Program

Output program menunjukkan bahwa deklarasi pertama `x` berhasil dimasukkan ke symbol table, sedangkan deklarasi kedua `x` ditandai sebagai redeclaration error.

```

1 ProgramNode(name: 'RedeclareTest') -> tab_index:37, lev:0
2 Declarations
3   VarDecls
4     VarDecl(x) -> tab_index:38, type:integer, ref:0, lev:0
5     VarDecl(x) -> tab_index:0, type:real, ref:0, lev:0,
6                   error:redclaration
7
8 Compound -> block_index:1, lev:1
9   Statements
10    Assign -> type:void
11    Target
12      Var(x) -> tab_index:38, type:integer, lev:0
13    Value
14      Int(10) -> type:integer

```

Bagian symbol table `tab` yang relevan adalah sebagai berikut.

idx	identifier	obj	type	ref	lev	adr	init
37	redeclaretest	program	void	0	0	0	yes
38	x	variable	integer	0	0	0	yes

Bagian `btab` yang relevan adalah sebagai berikut.

idx	last	lpar	psze	vsze
0	38	0	0	1
1	0	0	0	0

Semantic diagnostics yang dihasilkan adalah sebagai berikut.

```

1 Semantic error: redeclaration in same scope: x

```

5.3.4 Hasil Pengujian

Semantic analyzer berhasil:

1. memasukkan deklarasi pertama `x` sebagai `variable` bertipe `integer`;
2. mendeteksi deklarasi kedua `x` sebagai redeclaration pada scope yang sama;
3. memberi anotasi `tab_index:0` dan `error:redclaration` pada deklarasi kedua;
4. mempertahankan binding `x` yang valid pada `tab_index:38`;
5. memastikan assignment `x := 10` tetap mengacu pada deklarasi pertama yang valid;
6. menghasilkan semantic error yang sesuai.

Berdasarkan hasil tersebut, Test Case 4 dinyatakan **berhasil** karena semantic analyzer dapat mendeteksi deklarasi ulang identifier pada scope yang sama.

5.4 Test Case 5: Array dan Indexing

5.4.1 Tujuan Pengujian

Menguji kemampuan semantic analyzer dalam menangani deklarasi array, pencatatan informasi array pada `atab`, serta pengecekan tipe pada akses elemen array.

5.4.2 Input Program

```

1 program ArrayTest;
2
3 var
4     data : array [1..5] of integer;
5
6 begin
7     data[2] := 10;
8 end.
```

5.4.3 Output Program

Output program menunjukkan bahwa variabel `data` dikenali sebagai array. Ketika elemen `data[2]` digunakan sebagai target assignment, semantic analyzer menentukan bahwa tipe elemen tersebut adalah `integer`.

```

1 ProgramNode(name: 'ArrayTest') -> tab_index:37, lev:0
2 Declarations
3   VarDecls
4     VarDecl(data) -> tab_index:38, type:array, ref:0, lev:0
5
6 Compound -> block_index:1, lev:1
7   Statements
8     Assign -> type:void
9     Target
10      Var(data)[2] -> tab_index:38, type:integer, lev:0
11     Value
12      Int(10) -> type:integer
```

Bagian symbol table `tab` yang relevan adalah sebagai berikut.

idx	identifier	obj	type	ref	lev	adr	init
37	arraytest	program	void	0	0	0	yes
38	data	variable	array	0	0	0	yes

Bagian `btab` yang relevan adalah sebagai berikut.

idx	last	lpar	psze	vsze
0	38	0	0	5
1	0	0	0	0

Bagian `atab` yang relevan adalah sebagai berikut.

idx	xtyp	etyp	eref	low	high	elsz	size
0	subrange	integer	0	1	5	1	5

Semantic diagnostics yang dihasilkan adalah sebagai berikut.

```
1 No semantic errors.
```

5.4.4 Hasil Pengujian

Semantic analyzer berhasil:

1. mengenali deklarasi **data** sebagai variabel bertipe **array**;
2. mencatat referensi array pada **tab** melalui **ref:0**;
3. mencatat informasi array pada **atab**, yaitu index bertipe **subrange** dengan batas **1..5**;
4. mencatat tipe elemen array sebagai **integer**;
5. menghitung ukuran array sebesar 5 elemen pada **btap** melalui nilai **vsze = 5**;
6. menentukan bahwa akses **data[2]** menghasilkan nilai bertipe **integer**;
7. menentukan bahwa assignment **data[2] := 10** valid karena target dan value sama-sama bertipe **integer**;
8. menyelesaikan semantic analysis tanpa error.

Berdasarkan hasil tersebut, Test Case 5 dinyatakan **berhasil** karena semantic analyzer dapat menangani deklarasi array, indexing, dan pencatatan informasi array pada **atab**.

5.5 Test Case 7: Record dan Akses Field

5.5.1 Tujuan Pengujian

Menguji kemampuan semantic analyzer dalam menangani deklarasi tipe **record**, pencatatan field pada symbol table, deklarasi variabel bertipe record, serta pengecekan tipe pada akses field record.

5.5.2 Input Program

```
1 program RecordTest;
2
3 type
4     Point == record
5         x : integer;
6         y : integer;
7     end;
8
9 var
10     p : Point;
11
12 begin
```

```

13   p.x := 10;
14   p.y := p.x + 5;
15 end.

```

5.5.3 Output Program

Output program menunjukkan bahwa tipe `Point` dikenali sebagai record. Field `x` dan `y` dicatat sebagai field bertipe `integer`, sedangkan variabel `p` dikenali sebagai variabel bertipe record.

```

1 ProgramNode(name: 'RecordTest') -> tab_index:37, lev:0
2 Declarations
3   TypeDecls
4     TypeDecl(Point) -> tab_index:40, type:record, ref:1, lev:0
5   VarDecls
6     VarDecl(p) -> tab_index:41, type:record, ref:1, lev:0
7
8 Compound -> block_index:2, lev:1
9   Statements
10    Assign -> type:void
11    Target
12      Var(p).x -> tab_index:41, type:integer, lev:0
13    Value
14      Int(10) -> type:integer
15
16    Assign -> type:void
17    Target
18      Var(p).y -> tab_index:41, type:integer, lev:0
19    Value
20      BinOp(plus) -> type:integer
21      Var(p).x -> tab_index:41, type:integer, lev:0
22      Int(5) -> type:integer

```

Bagian symbol table `tab` yang relevan adalah sebagai berikut.

idx	identifier	obj	type	ref	lev	adr	init
37	recordtest	program	void	0	0	0	yes
38	x	field	integer	0	1	0	yes
39	y	field	integer	0	1	1	yes
40	point	type	record	1	0	0	yes
41	p	variable	record	1	0	0	yes

Bagian `btab` yang relevan adalah sebagai berikut.

idx	last	lpar	psze	vsze
0	41	0	0	2
1	39	0	0	2
2	0	0	0	0

Semantic diagnostics yang dihasilkan adalah sebagai berikut.

```

1 No semantic errors.

```

5.5.4 Hasil Pengujian

Semantic analyzer berhasil:

1. mengenali `Point` sebagai tipe bentukan berupa `record`;
2. mencatat field `x` dan `y` sebagai field bertipe `integer`;
3. mencatat variabel `p` sebagai variabel bertipe `record`;
4. menghubungkan tipe `Point` dan variabel `p` melalui referensi record yang sama, yaitu `ref:1`;
5. menentukan bahwa akses `p.x` menghasilkan tipe `integer`;
6. menentukan bahwa akses `p.y` menghasilkan tipe `integer`;
7. menentukan bahwa expression `p.x + 5` bertipe `integer`;
8. menyelesaikan semantic analysis tanpa error.

Berdasarkan hasil tersebut, Test Case 7 dinyatakan **berhasil** karena semantic analyzer dapat menangani deklarasi record, pencatatan field, dan akses field record dengan benar.

5.6 Test Case 9: Procedure dengan Parameter

5.6.1 Tujuan Pengujian

Menguji kemampuan semantic analyzer dalam menangani deklarasi procedure, parameter formal, scope procedure, akses variabel global dari dalam procedure, serta pengecekan tipe argumen pada pemanggilan procedure.

5.6.2 Input Program

```
1 program ProcedureParamTest;  
2  
3 var  
4     x : integer;  
5  
6 procedure SetX(v : integer);  
7 begin  
8     x := v;  
9 end;  
10  
11 begin  
12     SetX(10);  
13 end.
```

5.6.3 Output Program

Output program menunjukkan bahwa procedure **SetX** berhasil dicatat pada symbol table sebagai procedure bertipe **void**. Parameter **v** dicatat sebagai parameter bertipe **integer** pada level scope procedure.

```

1 ProgramNode(name: 'ProcedureParamTest') -> tab_index:37, lev:0
2 Declarations
3   VarDecls
4     VarDecl(x) -> tab_index:38, type:integer, ref:0, lev:0
5
6     ProcedureDecl(SetX) -> tab_index:39, block_index:1, lev:0, type:void
7       Parameters
8         Param(v) -> tab_index:40, type:integer, ref:0, lev:1
9
10      Block -> block_index:1, lev:1
11        Compound
12          Statements
13            Assign -> type:void
14              Target
15                Var(x) -> tab_index:38, type:integer, lev:0
16              Value
17                Var(v) -> tab_index:40, type:integer, lev:1
18
19      Compound -> block_index:2, lev:1
20        Statements
21          Call(SetX) -> tab_index:39, lev:1
22            Args
23              Int(10) -> type:integer

```

Bagian symbol table **tab** yang relevan adalah sebagai berikut.

1	idx	identifier	obj	type	ref	lev	adr	init	params
2	-----								
3	37	procedureparamtest	program	void	0	0	0	yes	
4	38	x	variable	integer	0	0	0	yes	
5	39	setx	procedure	void	1	0	0	yes	
		integer							
6	40	v	parameter	integer	0	1	0	yes	

Bagian **btabs** yang relevan adalah sebagai berikut.

1	idx	last	lpar	psze	vsze
2	-----				
3	0	39	0	0	1
4	1	40	40	1	0
5	2	0	0	0	0

Semantic diagnostics yang dihasilkan adalah sebagai berikut.

```

1 No semantic errors.

```

5.6.4 Hasil Pengujian

Semantic analyzer berhasil:

1. memasukkan variabel global **x** sebagai **variable** bertipe **integer**;
2. memasukkan procedure **SetX** sebagai **procedure** bertipe **void**;
3. mencatat parameter **v** sebagai **parameter** bertipe **integer**;
4. membedakan scope global dan scope procedure melalui nilai **lev**;
5. mengizinkan body procedure mengakses variabel global **x**;
6. memastikan assignment **x := v** valid karena target dan value sama-sama bertipe **integer**;
7. mencatat signature parameter procedure **SetX** sebagai **integer**;
8. memastikan pemanggilan **SetX(10)** valid karena argumen **10** bertipe **integer**;
9. menyelesaikan semantic analysis tanpa error.

Berdasarkan hasil tersebut, Test Case 9 dinyatakan **berhasil** karena semantic analyzer dapat menangani procedure, parameter formal, scope procedure, dan pengecekan argumen procedure dengan benar.

5.7 Analisis Hasil Pengujian

Berdasarkan hasil pengujian, semantic analyzer berhasil menjalankan fungsi utama yang dibutuhkan pada Milestone 3. Pada program valid, semantic analyzer dapat membentuk Decorated AST, mengisi symbol table, dan menyelesaikan proses analisis tanpa menghasilkan semantic error.

Pada pengujian deklarasi dan scope, semantic analyzer berhasil mencatat identifier ke dalam **tab**, membedakan lexical level, serta melakukan lookup identifier dari scope terdalam ke scope terluar. Hal ini terlihat pada pengujian procedure, ketika body procedure dapat mengakses variabel global dan parameter lokal secara benar.

Pada pengujian type checking, semantic analyzer berhasil menentukan tipe literal, variabel, dan expression. Assignment valid dapat diterima, sedangkan assignment dengan tipe yang tidak compatible menghasilkan semantic error. Operator dan kondisi statement juga diperiksa berdasarkan tipe operand yang digunakan.

Pada pengujian structured type, semantic analyzer berhasil menangani array dan record. Informasi array dicatat pada **atab**, sedangkan field record dicatat sebagai identifier dengan objek **field** dan dihubungkan melalui **btabs**. Akses field yang valid diterima, sedangkan akses field yang tidak terdaftar menghasilkan semantic error.

Secara keseluruhan, seluruh test case menghasilkan keluaran sesuai ekspektasi. Dengan demikian, semantic analyzer telah mampu melakukan pembentukan Decorated AST, pengelolaan symbol table, scope checking, type checking, dan pelaporan semantic error.

6 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, semantic analyzer berhasil dibangun untuk memproses Abstract Syntax Tree menjadi Decorated AST. Semantic analyzer dapat membentuk symbol table yang terdiri dari `tab`, `btab`, dan `atab`, serta memberikan anotasi semantic seperti `tab_index`, `type`, `ref`, dan `lev` pada node-node AST.

Hasil pengujian menunjukkan bahwa semantic analyzer mampu menangani deklarasi variabel, tipe dasar, array, record, procedure dengan parameter, scope checking, serta type checking pada assignment dan expression. Semantic analyzer juga berhasil mendeteksi beberapa semantic error, seperti redeclaration, type mismatch, kondisi `if` yang bukan boolean, dan akses field record yang tidak valid.

Dengan demikian, implementasi Milestone 3 telah memenuhi tujuan utama, yaitu membentuk Decorated AST, membangun symbol table, dan melakukan pemeriksaan semantic terhadap program masukan.

A Lampiran

A.1 Grammar dan Semantic Action

Lampiran ini berisi ringkasan grammar dan semantic action yang digunakan pada proses semantic analysis.

```

1  <program> ::= program ident ; <declaration-part> <compound-statement> .
2
3  Semantic action:
4  - Membuat ProgramNode.
5  - Membuat entry program pada symbol table.
6  - Membentuk block utama program.
7
8  <var-declaration> ::= var <identifier-list> : <type> ;
9
10 Semantic action:
11 - Memasukkan setiap identifier ke tab sebagai variable.
12 - Menyimpan tipe variable.
13 - Mendeteksi redeclaration pada scope yang sama.
14
15 <assignment-statement> ::= <variable> := <expression>
16
17 Semantic action:
18 - Mengecek apakah variable sudah dideklarasikan.
19 - Mengecek kesesuaian tipe target dan value.
20 - Memberi anotasi type pada node Assign.

```

A.2 Struktur AST

Lampiran ini berisi struktur node utama yang digunakan pada Abstract Syntax Tree.

```

1  ProgramNode
2  - name
3  - declarations
4  - main_block
5
6  VarDeclNode
7  - name
8  - type
9
10 TypeDeclNode
11 - name
12 - type
13
14 AssignNode
15 - target
16 - value
17
18 VarNode
19 - name
20 - selectors
21
22 BinOpNode
23 - op
24 - left

```

```

25 - right
26
27 ProcDeclNode
28 - name
29 - params
30 - body
31
32 ParamNode
33 - name
34 - type

```

A.3 Contoh Decorated AST

Lampiran ini berisi contoh Decorated AST hasil semantic analysis.

```

1 ProgramNode(name: 'Hello') -> tab_index:37, lev:0
2 Declarations
3   VarDecls
4     VarDecl(a) -> tab_index:38, type:integer, ref:0, lev:0
5     VarDecl(b) -> tab_index:39, type:integer, ref:0, lev:0
6
7 Compound -> block_index:1, lev:1
8   Statements
9     Assign -> type:void
10    Target
11      Var(a) -> tab_index:38, type:integer, lev:0
12    Value
13      Int(5) -> type:integer
14
15    Assign -> type:void
16    Target
17      Var(b) -> tab_index:39, type:integer, lev:0
18    Value
19      BinOp(plus) -> type:integer
20      Var(a) -> tab_index:38, type:integer, lev:0
21      Int(10) -> type:integer

```

A.4 Contoh Symbol Table Lengkap

Lampiran ini berisi contoh symbol table yang dihasilkan oleh semantic analyzer.

```

1 === SYMBOL TABLE: tab ===
2 idx  identifier  obj      type      ref  lev  adr  init
3 -----
4 36   writeln     procedure void      0    0   0   yes
5 37   hello       program  void      0    0   0   yes
6 38   a           variable integer    0    0   0   yes
7 39   b           variable integer    0    0   1   yes
8
9 === SYMBOL TABLE: btab ===
10 idx  last  lpar  psze  vsze
11 -----
12 0     39   0     0     2
13 1     0    0     0     0
14

```

```

15 === SYMBOL TABLE: atab ===
16 idx  xtyp      etyp      eref  low  high  elsz  size
17 -----
18 0      subrange  integer  0     1    5     1    5

```

A.5 Pranala Ke Repositori

<https://github.com/mhps-null/MDC-Tubes-IF2224-2026>

A.6 Pembagian Tugas

NIM	Nama	Tugas
13524041	Nathan Adhika Santosa	Source Code
13524053	Muhammad Haris Putra Sulastianto	Laporan, Source Code
13524085	Ariel Cornelius Sitorus	Source Code
13524091	Vara Azzara Ramli Pulukadang	Source Code

Table 4: Pembagian Tugas

Referensi

- [1] National University of Singapore. Abstract syntax tree, n.d. Diakses: 19 Mei 2026. URL: <https://nus-cs3203.github.io/course-website/contents/basic-spa-requirements/abstract-syntax-tree.html>.
- [2] Ruslan Spivak. Let's build a simple interpreter, part 7, n.d. Diakses: 19 Mei 2026. URL: <https://ruslanspivak.com/lrbasi-part7>.
- [3] Université de Liège. Semantic analysis, 2015. Diakses: 20 Mei 2026. URL: <https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>.
- [4] Amal Zouaq, Michel Gagnon, and Benoit Ozell. The semantic analysis pipeline, 2010. Figure from *Semantic Analysis Using Dependency-Based Grammars and Upper-Level Ontologies*. Diakses: 22 Mei 2026. URL: https://www.researchgate.net/figure/The-Semantic-Analysis-Pipeline_fig1_228947628.